

Control and Intelligent Systems

Philippe Dreesen Başak Sakçak

Department of Advanced Computing Sciences
Maastricht University, Netherlands

Fall 2026 (P1)

version September 21, 2026

Contents

1	Linear dynamical systems	3
1.1	Dynamics and memory	3
1.2	Differential and difference equations	3
1.3	Example: The mass-spring-damper system	5
1.4	Linear and nonlinear systems	7
1.5	Dynamics and time invariance	9
1.6	Further reading	9
2	Stability of linear systems	10
2.1	Equilibrium points	10
2.2	Stability	10
2.3	Continuous-time systems	11
2.4	Discrete-time systems	13
2.5	Summary	14
3	Linearization of a nonlinear system	15
3.1	Continuous-time systems	15
3.2	Discrete-time systems	16
3.3	Examples	17
4	State-space models and transformations	19
4.1	State-space and transfer-function models	19
4.2	Changing the state coordinates	21

5	Canonical state-space forms	23
5.1	Controller canonical form	23
5.2	Transforming a sytem into controller canonical form	24
5.3	Other canonical forms	26
6	Controllability, state feedback and eigenvalue placement	28
6.1	Controllability	28
6.2	State feedback and pole placement	29
6.3	Example: direct coefficient matching	30
6.4	Feedback in controller canonical coordinates	31
6.5	Constant reference tracking	32
7	Observability and full state observers	33
7.1	Can we estimate the state?	33
7.2	Observability as a linear system	34
7.3	A running state observer	35
7.4	Observer pole placement	35
7.5	Example: coefficient matching	36
7.6	Estimated-state feedback and the Separation Principle	36
8	Optimal Control	38
8.1	Controller performance	38
8.2	Finite-horizon optimal control	38
8.2.1	Example: Optimal control problem	39
8.2.2	Dynamic Programming	40
8.2.3	Example: Finite state/control spaces DP	42
8.3	Dynamic Programming in Continuous Spaces	44
8.4	Linear Quadratic Problems	46
8.4.1	Example: Scalar case	46
8.4.2	Linear Quadratic Regulator	49
8.5	Infinite-horizon optimal control	51
8.5.1	Infinite-horizon LQR	52

1 Linear dynamical systems

1.1 Dynamics and memory

A dynamical system is a system whose condition evolves over time. Its present condition contains the relevant information about the past required to determine what happens next. In this sense, one can think of a dynamical system as having *memory*. In general, dynamic effects arise whenever energy, material, or information accumulates or persists.

Mechanical or electrical systems are often used to introduce examples of dynamics, but dynamics is not limited to such systems. In physical systems, this memory is often associated with stored energy: for example, a moving mass stores kinetic energy, a compressed spring stores potential energy, a capacitor stores electrical energy, and a building stores heat. Similarly, a tank contains material, a battery stores charge, a population carries over from one generation to the next, an epidemic depends on the current numbers of susceptible and infected individuals, an inventory or queue records the cumulative difference between inflow and outflow, and internal variables in an algorithm keep track of a solution being optimized.

The state of a system

We call the quantities needed to predict the future evolution the *state* of the system. In dynamical systems, the present values of the state variables summarize relevant aspects of the past and determine the future evolution of the system, given the future input variables.

1.2 Differential and difference equations

The two main modeling paradigms for dynamical systems that we consider are differential equations, when we model the dynamics in continuous time, and difference equations, when we model the dynamics in discrete time.

Differential equations express how the state $x(t)$ changes by relating its value at time t to its rate of change $dx(t)/dt = \dot{x}(t)$, for example as in

$$\dot{x}(t) = f(x(t))$$

or

$$\ddot{x}(t) = g(x(t), \dot{x}(t)).$$

Here, we use the Newton notation $\dot{x}$ for the time derivative dx/dt . Provided with suitable initial conditions, such as $x(t_0) = x_0$ and $\dot{x}(t_0) = v_0$, the

differential equation can be solved for a function $x(t)$. A solution $x(t)$ is a trajectory describing the evolution of the system over time. Typically, we consider $t \in \mathbb{R}$ or $t \in \mathbb{R}^+$.

Difference equations, on the other hand, describe how a state variable is updated at discrete time instances k , for example

$$x[k+1] = f(x[k]),$$

or

$$y[k] = f(y[k-1], y[k-2], y[k-3]).$$

Together with a suitable set of initial conditions, such as $x[0] = x_0, x[1] = x_1, \dots$, the sequence $\{x[k]\}$ can be determined.

Remark 1.1. Notice that we distinguish the notation of discrete-time and continuous-time signals by using square brackets for the value $x[k]$ at discrete time $k \in \mathbb{Z}$ (or $k \in \mathbb{N}$), as opposed to parentheses for the value $x(t)$ at continuous time $t \in \mathbb{R}$.

Remark 1.2. We assume that the differential and difference equations considered here have a solution and that this solution is unique.

Remark 1.3. More general models can also be considered, for example, when the function f depends explicitly on time, but for the sake of simplicity, we typically do not consider such cases here.

Solution of the differential equation $\dot{x} = ax$

Consider the simple one-dimensional differential equation

$$\dot{x} = ax,$$

where $x \in \mathbb{R}$ and the initial condition is given as $x(0) = x_0$. The solution of this differential equation can be determined analytically. For $x \neq 0$, we have

$$\frac{\dot{x}}{x} = a, \quad \text{so} \quad \frac{dx}{x} = a dt,$$

and by integrating both sides, we find

$$\log |x| = at + C.$$

Using the initial condition $x(0) = x_0$ to determine the integration constant C gives the solution

$$x(t) = x_0 e^{at}.$$

This expression includes the solution $x(t) \equiv 0$ obtained when $x_0 = 0$. Notice that $x(t)$ converges to zero whenever $a < 0$ if $a \in \mathbb{R}$; when a is complex-valued, the convergence condition reduces to $\text{Re}(a) < 0$.

Solution of the difference equation $x[k + 1] = ax[k]$

Consider the simple one-dimensional difference equation

$$x[k + 1] = ax[k],$$

where $x[k] \in \mathbb{R}$ and the initial condition is given as $x[0] = x_0$. The solution of this difference equation can be determined by repeatedly applying the equation. We have

$$x[1] = ax[0] = ax_0$$

and

$$x[2] = ax[1] = a^2x_0,$$

and so on. Continuing in the same way, we find the solution

$$x[k] = a^k x_0.$$

The sequence $\{x[k]\}$ converges to zero when $|a| < 1$, where $|\cdot|$ denotes the (complex) magnitude.

1.3 Example: The mass-spring-damper system

Consider a mass m that moves horizontally on a frictionless platform. Its displacement from the equilibrium position of the unforced system is denoted by $x(t)$ (where a displacement to the right corresponds to $x > 0$). A spring pulls the mass back towards the equilibrium position with force $F_{\text{spring}} = -kx$, and a damper opposes the motion of the mass with force $F_{\text{damper}} = -c\dot{x}$. An external force, labeled $F(t)$ in Figure 1, serves as the input and is denoted by $u(t) = F(t)$.

In a control context, the input u may be chosen or manipulated by an actuator to influence the motion. The displacement x is a possible output of the system. The state of the system, however, will require not only x but also $\dot{x}$, as we will see.

To derive the dynamic equations governing the movement of the mass, we consider the force balance acting on the mass. The force balance and Newton's second law result in the dynamical equation (Figure 1)

$$m\ddot{x} = -kx - c\dot{x} + u, \quad \text{or} \quad m\ddot{x} + c\dot{x} + kx = u.$$

To reason about the solution of this differential equation, we first consider the unforced system, for which $u(t) \equiv 0$. The motion of the mass depends

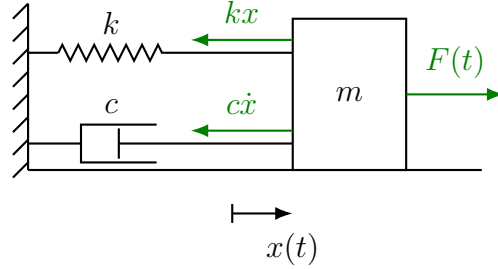


Figure 1: A mass-spring-damper system on a frictionless horizontal platform. The spring exerts a restoring force $-kx$, where $x(t)$ is the displacement from equilibrium, while the damper exerts a force $-c\dot{x}$ opposing the velocity. An external force $F(t)$ may be applied to the mass.

on two initial conditions: the initial displacement $x_0 = x(0)$ and the initial velocity $v_0 = \dot{x}(0)$. Notice that, even without an external force, the mass may continue to move because an initial displacement or velocity supplies potential or kinetic energy. The damper gradually dissipates this energy until the mass returns to the equilibrium position.

Exponential modes

The unforced mass-spring-damper equation is a second-order linear differential equation. As for the first-order case ($\dot{x} = ax$), it turns out that exponential functions are the expected solutions of linear differential equations. Sums of exponentials typically arise as solutions of linear differential equations; the reason is that differentiation preserves their form.

Recall that a sinusoid can be expressed as a sum of complex exponentials through Euler's identity $e^{j\theta} = \cos \theta + j \sin \theta$, so for example, $\cos(\omega t) = 1/2 (e^{j\omega t} + e^{-j\omega t})$.

Consider the differential equation $m\ddot{x} + c\dot{x} + kx = u$ from the mass-spring-damper system, and let us assume an exponential solution $x(t) = e^{\lambda t}$, with an as-yet unknown value of λ . Substituting this expression into the unforced mass-spring-damper equation reduces the differential equation to a polynomial equation in λ , called the characteristic equation:

$$p(\lambda) = m\lambda^2 + c\lambda + k = 0.$$

Each root λ_i of $p(\lambda)$ (that is, each solution of $p(\lambda) = 0$) produces an elementary exponential motion $e^{\lambda_i t}$. Recall that, before the initial conditions are imposed, a homogeneous differential equation has a family of solutions.

Because the equation is linear, linear combinations of solutions are again solutions, generally corresponding to different initial conditions. If the two roots λ_1 and λ_2 are distinct, the general solution is therefore

$$x(t) = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t},$$

with as-yet undetermined constants C_1 and C_2 . Once $x(0)$ and $\dot{x}(0)$ are specified, the constants C_1 and C_2 are uniquely defined — in this case they are determined by the initial displacement $x(0) = x_0$ and velocity $\dot{x}(0) = v_0$.

Remark 1.4. If the two roots of $p(\lambda)$ coincide, a modification of the expression $x(t) = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t}$ is required, but we do not consider that case here.

Remark 1.5. For the undamped oscillator, we have $c = 0$. If we let $m = k = 1$, for convenience, then the differential equation becomes

$$\ddot{x} + x = 0,$$

for which it can be reasoned that the solution $x(t)$ takes the form

$$x(t) = A \cos t + B \sin t.$$

In particular, $x(t) = \cos t$ describes a mass initially displaced by one unit and released at zero velocity: $x(0) = 1$ and $\dot{x}(0) = 0$.

Characterizing solutions for second-order differential equations

Let us consider the general case of a second-order differential equation with real-valued coefficients, with a corresponding characteristic polynomial $p(\lambda)$ of degree two. For distinct roots $\lambda_1 \neq \lambda_2$, we distinguish two cases: either both roots are real or they form a complex-conjugate pair. Let us assume that λ_1 and λ_2 are in the stable region (that is, their real part is negative). Real negative roots produce a decaying motion (without oscillation). Complex roots with a negative real part produce a damped oscillation of the form

$$x(t) = e^{-\sigma t} (A \cos(\omega t) + B \sin(\omega t)), \quad \sigma > 0.$$

The exponential factor describes the decay, while the sine and cosine describe the oscillation.

1.4 Linear and nonlinear systems

The mass–spring–damper equation is linear in the input u and in x , $\dot{x}$, and $\ddot{x}$: these quantities appear only to the first power, are not multiplied by one another, and do not occur inside nonlinear functions.

Linearity gives rise to the *principle of superposition*. Suppose that, under zero initial conditions, an input $u_1(t)$ produces an output $y_1(t)$ and an input $u_2(t)$ produces an output $y_2(t)$. For a linear system, the combined input $\alpha u_1(t) + \beta u_2(t)$ then produces the combined output $\alpha y_1(t) + \beta y_2(t)$:

$$u_1 \longrightarrow y_1, \quad u_2 \longrightarrow y_2, \quad \alpha u_1 + \beta u_2 \longrightarrow \alpha y_1 + \beta y_2.$$

The condition of zero initial conditions is important because it isolates the response caused by the input.

The same principle applies to free motion. When $u(t) \equiv 0$, if $x_1(t)$ and $x_2(t)$ are possible motions resulting from two sets of initial conditions, then $\alpha x_1(t) + \beta x_2(t)$ is the motion resulting from the corresponding linear combination of those initial conditions. This is precisely the property used above to combine the two elementary exponential modes.

The principle of superposition is a very powerful one: it enables a ‘modular viewpoint’ in which we can break a system into simpler pieces, solve each of them separately and then recombine the parts. More generally, the complete motion of a linear system can be separated into a free response caused by the initial conditions and a forced response caused by the input.

Superposition does not generally hold for nonlinear systems. Consider the simple static relation $y = u^2$. Doubling the input gives $(2u)^2 = 4u^2$, rather than twice the original output, while adding two inputs gives

$$(u_1 + u_2)^2 = u_1^2 + 2u_1u_2 + u_2^2,$$

which contains an additional interaction term u_1u_2 . Scaling and addition therefore do not pass independently through the system.

The same issue appears in nonlinear dynamics. A nonlinear spring might be characterized by a nonlinear differential equation as

$$m\ddot{x} + c\dot{x} + kx + k_3x^3 = 0.$$

Combining two motions produces additional cross-terms through the cubic term. So, even if $x_1(t)$ and $x_2(t)$ are solutions, then $x_1(t) + x_2(t)$ is generally not. Some nonlinear equations can be solved analytically, but there is no general construction comparable to the linear case; qualitative and numerical methods are often needed.

In many practical cases, the system of interest is nonlinear. In such cases, we may still benefit from the nice properties of linear systems by considering a small excursion of the signals around some point of interest, like an equilibrium point, where a linear approximation often works sufficiently well.

1.5 Dynamics and time invariance

A system is called *time invariant* if its governing laws do not depend on the absolute time at which an experiment is performed. Dynamics and time invariance are different ideas. A dynamical system has memory and produces trajectories that evolve over time. There is a lot of (control) results that concern ‘linear time-invariant’ (LTI) dynamical systems, as they benefit from the convenient properties we have discussed above.

The mass-spring-damper system is a linear time-invariant system. It is dynamic because its future motion depends on its present displacement and velocity; it is described by a differential equation, and it exhibits a ‘memory’ in the form of the energy stored in the system. It is time invariant because m , c , and k are constants and is not dependent on time t : the same physical law applies whenever the experiment begins.

We emphasize that a trajectory $x(t)$ may therefore change continually even though the system itself is time invariant. ‘Time invariant’ does not mean that nothing changes with time; it means that the laws governing the change remain the same. Linear systems with time-dependent coefficients also exist, but they are outside the scope of this course.

1.6 Further reading

- M. Arcak and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

See Chapter 1, Sections A.1–A.2, B.1–B.2, and B.11–B.12, and Chapter 4, Sections A.1–A.2.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

See Sections 1.1–1.3, 3.1–3.4, and 6.1–6.3.

- S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, third edition (2024).

See Sections 1.2–1.3 and 5.1–5.2.

2 Stability of linear systems

2.1 Equilibrium points

Dynamical systems described by (non)linear differential or difference equations often possess points in the state space (specific values of the state variable) for which the governing equations dictate that the system should remain at that point. We call such points *equilibrium points* (or *fixed points*).

The conditions for equilibrium points are easy to understand, however, they take a different form when dealing with continuous-time and discrete-time systems. Equilibrium points for continuous-time systems are characterized by the equation

$$\dot{x} = 0,$$

so for the continuous-time linear system $\dot{x} = Ax$, an equilibrium point x_e therefore satisfies

$$Ax_e = 0.$$

For discrete-time systems the condition is

$$x[k+1] = x[k],$$

so for $x[k+1] = Ax[k]$, it satisfies

$$(A - I)x_e = 0.$$

In both cases, the origin $x = 0$ is always an equilibrium point. Other equilibria can occur when 0 is an eigenvalue of A in continuous time or when 1 is an eigenvalue of A in discrete time.

2.2 Stability

To study stability, we consider ‘autonomous’ linear systems (that is, systems without inputs) and study the stability of the equilibrium point at the origin. In both continuous and discrete time, the state $x = 0$ remains at $x = 0$.

In this chapter, we focus primarily on *asymptotic stability*. Informally, an equilibrium is *stable* if every trajectory that starts sufficiently close to it remains close, and it is *asymptotically stable* if, in addition, these trajectories converge to the equilibrium. For an asymptotically stable linear system, every trajectory converges to the origin. An equilibrium is *unstable* if it is not stable: trajectories can start arbitrarily close to it and nevertheless leave

a prescribed neighbourhood. So, for an asymptotically stable linear system, the origin is the single equilibrium point.

In the following, we discuss the continuous- and discrete-time cases separately. Nevertheless, their analyses have strong similarities: in both cases, the evolution of a scalar system is determined by one number, while the evolution of the multivariable system is determined by the set of eigenvalues of a matrix. The difference between the continuous-time and discrete-time stability conditions lies in the region of the complex plane in which these eigenvalues must lie.

2.3 Continuous-time systems

Scalar case

Consider first the scalar differential equation

$$\dot{x}(t) = ax(t),$$

with $x \in \mathbb{R}$, $a \in \mathbb{R}$, and the initial condition $x(0) = x_0$. Its solution is

$$x(t) = e^{at}x_0.$$

If $a < 0$, the exponential e^{at} decays and $x(t)$ converges to zero as time t grows. If $a > 0$, its magnitude grows without bound for every $x_0 \neq 0$. The boundary case $a = 0$ gives $x(t) = x_0$: the state remains constant and therefore does not converge to the origin unless it starts there. This case is stable but not asymptotically stable.

To prepare for the multivariable case, it is useful to extend the scalar calculation to consider $a \in \mathbb{C}$ and $x \in \mathbb{C}$. Since

$$|e^{at}| = e^{\operatorname{Re}(a)t},$$

it is the real part of a that determines whether the magnitude grows or decays. A nonzero imaginary part produces an oscillation. In particular, if $\operatorname{Re}(a) = 0$ and $\operatorname{Im}(a) \neq 0$, every nonzero solution oscillates persistently without decaying. This case is stable but not asymptotically stable. Thus, the scalar system is asymptotically stable precisely when $\operatorname{Re}(a) < 0$.

Multivariable case

Now consider the multivariable system

$$\dot{x}(t) = Ax(t),$$

where $x(t) \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. The simplest case occurs when A is diagonal:

$$A = \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix}.$$

The differential equation then consists of n independent scalar equations,

$$\dot{x}_i(t) = \lambda_i x_i(t), \quad x_i(t) = e^{\lambda_i t} x_i(0).$$

As in the scalar case, every state component converges to zero if and only if $\text{Re}(\lambda_i) < 0$ for every λ_i .

For a general matrix A , the solution of $\dot{x} = Ax$ can be expressed using the matrix exponential,

$$x(t) = e^{At} x(0), \quad \text{with} \quad e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots$$

To understand this solution, suppose that A has a complete eigenvalue decomposition

$$A = V\Lambda V^{-1},$$

where the columns of V are eigenvectors of A and Λ contains the corresponding eigenvalues on its diagonal. Introducing a transformed state $z = V^{-1}x$ gives

$$\dot{z} = V^{-1}\dot{x} = V^{-1}AVz = \Lambda z.$$

The eigenvector coordinates therefore decouple the system into scalar equations. Equivalently,

$$e^{At} = V e^{\Lambda t} V^{-1}.$$

Each eigenvalue produces a mode $e^{\lambda_i t}$ along its corresponding eigenvector direction. Consequently, determining whether the system $\dot{x} = Ax$ is asymptotically stable boils down to the condition that $\text{Re}(\lambda_i) < 0$ for all eigenvalues of A . The eigenvalues must therefore lie strictly in the open left half-plane. An eigenvalue in the open right half-plane produces a growing mode. Under the diagonalizability assumption used here, eigenvalues on the imaginary axis produce modes that do not decay: a zero eigenvalue gives a constant mode, while a nonzero purely imaginary pair produces a persistent oscillation. If

all other eigenvalues lie in the open left half-plane, such boundary cases are stable but not asymptotically stable.

The fact that we discuss the occurrence of complex eigenvalues is not merely for mathematical completeness. The case of complex eigenvalues prepares us for the multivariable case: A real-valued matrix may have complex eigenvalues; however, its complex eigenvalues occur in conjugate pairs and combine to produce real oscillatory trajectories.

2.4 Discrete-time systems

Scalar case

The corresponding scalar discrete-time system is

$$x[k+1] = ax[k].$$

Repeated substitution gives

$$x[k] = a^k x[0].$$

Allowing $a \in \mathbb{C}$ and $x \in \mathbb{C}$, the magnitude of the solution satisfies

$$|x[k]| = |a|^k |x[0]|.$$

Hence, $x[k]$ converges to zero when $|a| < 1$ and grows in magnitude when $|a| > 1$, provided $x[0] \neq 0$. If $|a| = 1$, its magnitude does not decay. For example, $a = 1$ gives a constant trajectory, $a = -1$ produces an alternating trajectory, and a complex number on the unit circle produces a persistent oscillation. These boundary cases are stable but not asymptotically stable. Thus, the scalar system is asymptotically stable precisely when $|a| < 1$.

Multivariable case

For the multivariable system

$$x[k+1] = Ax[k],$$

the solution is

$$x[k] = A^k x[0].$$

When A is diagonal, the state components again evolve independently as

$$x_i[k] = \lambda_i^k x_i[0].$$

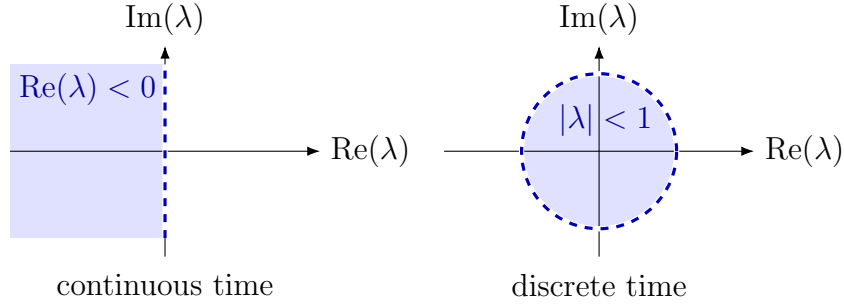


Figure 2: Regions of asymptotic stability for the eigenvalues of continuous- and discrete-time linear systems. The boundaries (dashed) are not included.

More generally, if $A = V\Lambda V^{-1}$ and $z = V^{-1}x$, then

$$z[k+1] = \Lambda z[k], \quad A^k = V\Lambda^k V^{-1}.$$

Each eigenvalue therefore produces a mode λ_i^k , and we obtain the stability condition that $x[k+1] = Ax[k]$ is asymptotically stable if and only if $|\lambda_i| < 1$ for all eigenvalues of A . Thus, all eigenvalues must lie strictly inside the unit circle in the complex plane. An eigenvalue outside the unit circle produces a growing mode. Under the diagonalizability assumption, an eigenvalue on the unit circle gives a mode that persists rather than decays. If all other eigenvalues lie strictly inside the unit circle, such a boundary case is stable but not asymptotically stable.

2.5 Summary

The two stability conditions are summarized in Table 1 and Figure 2.

Table 1: Stability conditions for continuous-time and discrete-time systems

system	modes	asymptotically stable if
$\dot{x} = Ax$	$e^{\lambda_i t}$	$\operatorname{Re}(\lambda_i) < 0$ for every i
$x[k+1] = Ax[k]$	λ_i^k	$ \lambda_i < 1$ for every i

Remark 2.1. The discussion above assumes that A has a complete set of eigenvectors and can therefore be diagonalized. This is not true for every matrix. In the general case, solutions need not be combinations of pure modes $e^{\lambda_i t}$ or λ_i^k ; additional polynomial factors may appear. In particular, without diagonalizability, an eigenvalue on the stability boundary may produce polynomial growth rather than a bounded persistent mode.

These more general forms are outside the scope of this course. Nevertheless, the strict asymptotic-stability conditions themselves remain unchanged: all continuous-time eigenvalues must have negative real parts, and all discrete-time eigenvalues must have magnitude smaller than one.

Further reading

- M. Arcaç and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

See Chapter 3, Sections C–D, and Chapter 4, Sections A.1–A.3 and C.1.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

See Chapter 6, especially Sections 6.1–6.3.

- S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, third edition (2024).

See Chapter 5, especially Sections 5.1–5.2, for a geometric treatment of continuous-time two-dimensional linear systems.

3 Linearization of a nonlinear system

As we have seen, understanding a dynamical system is convenient when it is a linear system. For nonlinear systems, we can often assume that the dynamics are dominated by linear dynamical effects, as long as the state variables are in the vicinity of an equilibrium point. We will now study how we can linearize a nonlinear system, so we obtain a linear state space model (either in continuous or discrete time).

3.1 Continuous-time systems

Consider a nonlinear system (continuous time)

$$\begin{aligned}\dot{x}(t) &= f(x(t), u(t)), \\ y(t) &= g(x(t), u(t)),\end{aligned}$$

where we are using Newton’s dot notation $\dot{x}(t) = dx(t)/dt$. The behavior of the system around an equilibrium point can be analyzed and modeled with

a linear model. We consider the equilibrium point (x_e, u_e) , with

$$\begin{aligned}\dot{x}(t) &= f(x_e, u_e) = 0, \\ y_e &= g(x_e, u_e).\end{aligned}$$

The Taylor series about the equilibrium point (x_e, u_e) is given as

$$\begin{aligned}\dot{x}(t) &\approx f(x_e, u_e) + \left[\frac{\partial f}{\partial x} \right] \Big|_{(x_e, u_e)} (x(t) - x_e) + \left[\frac{\partial f}{\partial u} \right] \Big|_{(x_e, u_e)} (u(t) - u_e), \\ y(t) &\approx g(x_e, u_e) + \left[\frac{\partial g}{\partial x} \right] \Big|_{(x_e, u_e)} (x(t) - x_e) + \left[\frac{\partial g}{\partial u} \right] \Big|_{(x_e, u_e)} (u(t) - u_e).\end{aligned}$$

Remark that the Jacobians are generally matrix expressions (and function of x and u), where the (i, j) -th entry in $[\partial f / \partial x]$ corresponds to $\partial f_i / \partial x_j$.

If we let $\tilde{x}(t) = x(t) - x_e$, $\tilde{u}(t) = u(t) - u_e$ and $\tilde{y}(t) = y(t) - y_e$, this can be written in a more familiar state-space formulation as

$$\begin{aligned}\dot{\tilde{x}}(t) &= A\tilde{x}(t) + B\tilde{u}(t) \\ \tilde{y}(t) &= C\tilde{x}(t) + D\tilde{u}(t),\end{aligned}$$

with

$$\begin{aligned}A &= \left[\frac{\partial f}{\partial x} \right] \Big|_{(x_e, u_e)}, & B &= \left[\frac{\partial f}{\partial u} \right] \Big|_{(x_e, u_e)}, \\ C &= \left[\frac{\partial g}{\partial x} \right] \Big|_{(x_e, u_e)}, & D &= \left[\frac{\partial g}{\partial u} \right] \Big|_{(x_e, u_e)},\end{aligned}$$

where $(\tilde{x}(t), \tilde{u}(t))$ and $\tilde{y}(t)$ refer to the excursions from the equilibrium point (x_e, u_e) and output $y_e = g(x_e, u_e)$. Remark that x_e , u_e and y_e do not depend on time.

3.2 Discrete-time systems

Linearizing about an equilibrium point is very similar for the discrete-time case. The most important distinction is in how an equilibrium point is defined: equilibrium points in discrete-time systems are characterized by $x[k+1] = x[k]$, which leads to defining the equilibrium state x_e as

$$\begin{aligned}x_e &= f(x_e, u_e), \\ y_e &= g(x_e, u_e).\end{aligned}$$

To linearize the system, we consider an excursion about an equilibrium point (x_e, u_e) as $x[k] = x_e + \tilde{x}[k]$, $u[k] = u_e + \tilde{u}[k]$, $y[k] = y_e + \tilde{y}[k]$.

From the Taylor series about (x_e, u_e) , we have

$$\begin{aligned} x[k+1] &\approx f(x_e, u_e) + \left[\frac{\partial f}{\partial x} \right] \Big|_{(x_e, u_e)} (x[k] - x_e) + \left[\frac{\partial f}{\partial u} \right] \Big|_{(x_e, u_e)} (u[k] - u_e), \\ y[k] &\approx g(x_e, u_e) + \left[\frac{\partial g}{\partial x} \right] \Big|_{(x_e, u_e)} (x[k] - x_e) + \left[\frac{\partial g}{\partial u} \right] \Big|_{(x_e, u_e)} (u[k] - u_e), \end{aligned}$$

which simplifies to¹

$$\begin{aligned} \dot{\tilde{x}}[k] &= A\tilde{x}[k] + B\tilde{u}[k] \\ \tilde{y}[k] &= C\tilde{x}[k] + D\tilde{u}[k], \end{aligned}$$

with

$$\begin{aligned} A &= \left[\frac{\partial f}{\partial x} \right] \Big|_{(x_e, u_e)}, & B &= \left[\frac{\partial f}{\partial u} \right] \Big|_{(x_e, u_e)}, \\ C &= \left[\frac{\partial g}{\partial x} \right] \Big|_{(x_e, u_e)}, & D &= \left[\frac{\partial g}{\partial u} \right] \Big|_{(x_e, u_e)}. \end{aligned}$$

3.3 Examples

Equilibria and Linearization of a Discrete Nonlinear Map In this example, we consider the problem of determining the equilibrium points of a discrete-time system and linearizing the dynamics around the equilibrium points. Consider

$$\begin{aligned} x_1[k+1] &= 2x_1[k] - x_1[k]x_2[k], \\ x_2[k+1] &= 3x_2[k] - 2x_1[k]x_2[k] + u[k], \end{aligned}$$

and $u[k] \equiv 2$.

Equilibria. An equilibrium (x_1, x_2) satisfies

$$\begin{aligned} x_1 &= 2x_1 - x_1x_2 \Rightarrow x_1(1 - x_2) = 0, \\ x_2 &= 3x_2 - 2x_1x_2 + 2 \Rightarrow x_2(1 - x_1) = -1. \end{aligned}$$

Hence $(x_1, x_2) \in \{(0, -1), (2, 1)\}$.

¹Remark that the term $f(x_e, u_e)$ is not zero as in the continuous-time case, but it cancels from the equation because the left-hand-side of the equation reads $x[k+1] = x_e + \tilde{x}[k+1]$ and the equilibrium property entails $x_e = f(x_e, u_e)$.

Linearization. Let $f_1 = 2x_1 - x_1x_2$, $f_2 = 3x_2 - 2x_1x_2 + u$. The Jacobians are

$$A(x) = \frac{\partial f}{\partial x} = \begin{bmatrix} 2 - x_2 & -x_1 \\ -2x_2 & 3 - 2x_1 \end{bmatrix}, \quad B = \frac{\partial f}{\partial u} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

So in general, the linearized model about an equilibrium is

$$\tilde{x}[k+1] = A(x)\tilde{x}[k] + B\tilde{u}[k].$$

At $(0, -1)$.

$$A = \begin{bmatrix} 3 & 0 \\ 2 & 3 \end{bmatrix}, \quad \text{eig}(A) = \{3, 3\}.$$

Both $|\lambda| > 1$ so the equilibrium is *unstable*.

At $(2, 1)$.

$$A = \begin{bmatrix} 1 & -2 \\ -2 & -1 \end{bmatrix}, \quad \text{eig}(A) = \{\sqrt{5}, -\sqrt{5}\}.$$

Both $|\lambda| > 1$ so the equilibrium is *unstable*.

Further reading

- M. Arcak and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

Chapter 1, Section C, especially Section C.4 on Jacobians and Taylor expansions.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

Section 6.4 on linearization.

- S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, third edition (2024).

Sections 2.4 and 6.3, for linearization near fixed points in continuous-time systems.

4 State-space models and transformations

4.1 State-space and transfer-function models

Two alternative modeling paradigms

Before we study state-space transformations, we first recall the connection between *state-space models* and *transfer functions*. Both can be used to describe linear dynamical systems, although they offer different viewpoints. A transfer function describes the input–output behaviour of a system, usually under zero initial conditions, whereas a state-space model explicitly represents its internal state.

Historically, transfer-function models have been closely associated with classical control, frequency-domain methods, signal processing, and telecommunications. Transfer functions describe the input-output behavior using complex variables (in the Laplace domain or frequency domain), and require a bit of complex function theory.

State-space models are central to modern control theory. State-space models describe the system dynamics by means of (coupled) first-order differential equations, and are hence a time-domain tool, in their standard form. They require somewhat more advanced linear-algebraic tools, but often allow for geometric intuition and they extend easily to systems with multiple inputs and outputs. They provide a powerful and flexible modeling framework.

Problem setup

Consider a linear time-invariant state-space system in continuous time,

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t),$$

or in discrete time,

$$x[k+1] = Ax[k] + Bu[k], \quad y[k] = Cx[k] + Du[k].$$

Here x contains n state variables. With m inputs and p outputs, the matrices have sizes $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$, $C \in \mathbb{R}^{p \times n}$, and $D \in \mathbb{R}^{p \times m}$.

The state captures the *memory* of the system: together with future inputs, its present value determines the future evolution and outputs. The relation between inputs and outputs can also be described directly by eliminating the state in the transform domain.

Continuous time. Write $X(s) = \mathcal{L}\{x(t)\}$, and similarly for $U(s)$ and $Y(s)$. Assuming a zero initial state, the transformed state equation gives

$$\begin{aligned} sX(s) &= AX(s) + BU(s), \\ X(s) &= (sI - A)^{-1}BU(s). \end{aligned}$$

Substitution into the output equation yields

$$Y(s) = H(s)U(s), \quad H(s) = C(sI - A)^{-1}B + D.$$

Discrete time. Using the one-sided Z-transform

$$X(z) = \mathcal{Z}\{x[k]\} = \sum_{k=0}^{\infty} x[k]z^{-k},$$

we have

$$\mathcal{Z}\{x[k+1]\} = zX(z) - zx[0].$$

For $x[0] = 0$, the same reasoning gives

$$Y(z) = H(z)U(z), \quad H(z) = C(zI - A)^{-1}B + D.$$

These expressions are defined for values of s or z for which $sI - A$ or $zI - A$, respectively, is invertible. For a single-input, single-output system, H is a scalar transfer function; with several inputs or outputs, it is a transfer matrix. The state no longer appears explicitly in this description: it has been eliminated from the input–output relation.

Remark 4.1. The relation $Y = HU$ describes the response from a zero initial state. A nonzero initial state produces an additional contribution:

$$\begin{aligned} Y(s) &= H(s)U(s) + C(sI - A)^{-1}x(0), \\ Y(z) &= H(z)U(z) + C(zI - A)^{-1}zx[0]. \end{aligned}$$

Remark 4.2. For a stable linear time-invariant system, a sinusoidal input with angular frequency ω produces, after any transient response has decayed, a sinusoidal output with the same frequency. The amplitude is multiplied by $|H(j\omega)|$ (magnitude), while the phase is shifted by $\arg H(j\omega)$ (angle). This leads to the *frequency response*, obtained by evaluating the transfer function along the imaginary axis:

$$H(j\omega) = H(s)|_{s=j\omega}.$$

This relation can be written as $H(j\omega) = Y(j\omega)/U(j\omega)$. For a discrete-time system, the frequency response is obtained by evaluating $H(z)$ on the unit circle, at $z = e^{j\omega}$.

4.2 Changing the state coordinates

The state coordinates contained in $x \in \mathbb{R}^n$ do not have a unique representation. For example, we may use different units, reorder the state variables, or replace them by invertible linear combinations. Such choices change the internal description without changing the system itself (for example, the transfer function).

Let $T \in \mathbb{R}^{n \times n}$ be a constant invertible matrix and define

$$z[k] = Tx[k], \quad x[k] = T^{-1}z[k],$$

where $z \in \mathbb{R}^n$ denotes the transformed state². Invertibility ensures that no state information is lost: each old state x corresponds to exactly one new state z , and conversely. Since T is constant, $z[k+1] = Tx[k+1]$. We have

$$\begin{aligned} z[k+1] &= T(Ax[k] + Bu[k]) = TAT^{-1}z[k] + TBu[k], \\ y[k] &= CT^{-1}z[k] + Du[k]. \end{aligned}$$

Thus, in the new coordinates, we find a state-space model with model matrices $\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D}$, where

$$\tilde{A} = TAT^{-1}, \quad \tilde{B} = TB, \quad \tilde{C} = CT^{-1}, \quad \tilde{D} = D.$$

The input u and output y retain their original meaning; only the state coordinates change. The initial state x_0 becomes $z_0 = Tx_0$.

Remark 4.3. The same formulas hold in continuous time when $z(t) = Tx(t)$.

The transfer function is unchanged

To treat continuous and discrete time simultaneously, let q denote the transform variable: $q = s$ in continuous time and $q = z$ in discrete time. The transfer function of the transformed model is

$$\tilde{H}(q) = CT^{-1}(qI - TAT^{-1})^{-1}TB + D.$$

Since

$$qI - TAT^{-1} = T(qI - A)T^{-1},$$

we have

$$(qI - TAT^{-1})^{-1} = T(qI - A)^{-1}T^{-1}.$$

² $z[k]$ should not be confused with the transform variable used in the z -transform

Consequently,

$$\begin{aligned}\tilde{H}(q) &= CT^{-1}T(qI - A)^{-1}T^{-1}TB + D \\ &= C(qI - A)^{-1}B + D = H(q).\end{aligned}$$

Notice that the transformation matrix cancels: the internal coordinates have changed, but the input-output relation has not. (This remains true for a nonzero initial state provided that it is transformed consistently as $z_0 = Tx_0$.)

Other preserved properties

The map $A \mapsto TAT^{-1}$ is called a *similarity transformation*. Similar matrices have the same characteristic polynomial because

$$\det(qI - TAT^{-1}) = \det(T(qI - A)T^{-1}) = \det(qI - A).$$

They therefore have the same eigenvalues, including algebraic multiplicities. A change of state coordinates cannot change the stability of the system.

Controllability and observability (see later) are also preserved. Indeed, their matrices transform as

$$\begin{aligned}\widetilde{W}_c &= TW_c, & W_c &= [B \quad AB \quad \cdots \quad A^{n-1}B], \\ \widetilde{W}_o &= W_o T^{-1}, & W_o &= \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}.\end{aligned}$$

Multiplication by an invertible matrix does not change rank. Consequently, a coordinate transformation cannot make an uncontrollable system controllable or an unobservable system observable.

Remark 4.4. Preservation of the eigenvalues and preservation of the transfer function are related but distinct statements. A nonminimal realization may contain modes that cannot be excited by the input or observed in the output. Their eigenvalues do not appear as transfer-function poles. In a minimal SISO realization, no such hidden modes are present, and the poles of the transfer function are precisely the eigenvalues of A , including their multiplicities.

For every invertible T , the realizations

$$(A, B, C, D) \quad \text{and} \quad (TAT^{-1}, TB, CT^{-1}, D)$$

therefore provide equivalent descriptions of the same system. This freedom can be used to select coordinates in which the system matrices have a particularly convenient structure.

Further reading

- M. Arcaç and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

See Chapter 1, especially Sections A.2, A.6, B.1–B.3, and B.7.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

See Sections 3.2, 6.3, 9.1–9.2, and 9.4.

5 Canonical state-space forms

A state-space realization is not unique: an invertible change of state coordinates produces different state-space matrices $\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D}$ that describe the same system. This freedom can be used to place the matrices in a convenient standard structure, called a *canonical form*. Different canonical forms are useful for different purposes. We first introduce the controller canonical form and then explain how a controllable realization can be transformed into it. Other common forms are stated more briefly.

Throughout this chapter, we simplify the notation by assuming that there is no direct input–output term, so $D = 0$. This is only a bookkeeping simplification: if a realization has $D \neq 0$, we can define $\bar{y} = y - Du = Cx$, apply the constructions below to the strictly proper mapping from u to $\bar{y}$, and finally reconstruct $y = \bar{y} + Du$. Notice that a change of state coordinates itself does not remove the direct term: it always leaves D unchanged.

5.1 Controller canonical form

Consider the monic polynomial

$$p(q) = q^n + a_1 q^{n-1} + a_2 q^{n-2} + \cdots + a_n.$$

The *controller canonical form*, also called the *controllable canonical form*, uses

$$A_c = \begin{bmatrix} -a_1 & -a_2 & \cdots & -a_{n-1} & -a_n \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The coefficients of $p(q)$ appear in the first row of A_c , and the input enters through the first state equation. The remaining equations form a chain between successive state variables of the form $x_2[k+1] = x_1[k]$, $x_3[k+1] = x_2[k]$, $\dots$, $x_n[k+1] = x_{n-1}[k]$. The characteristic polynomial of A_c is $p(q)$.

The output matrix C_c does not affect controllability and may take arbitrary values. When the form is constructed from the strictly proper SISO transfer function

$$H(q) = \frac{b_1 q^{n-1} + b_2 q^{n-2} + \dots + b_n}{q^n + a_1 q^{n-1} + \dots + a_n},$$

the remaining matrices are

$$C_c = [b_1 \quad b_2 \quad \dots \quad b_n], \quad D_c = 0.$$

Here and throughout this chapter, q may denote either the continuous-time variable s or the discrete-time variable z . The matrix construction is the same in both cases.

The name ‘controller canonical form’ follows from its controllability matrix of (A_c, B_c, C_c, D_c) , which is

$$\begin{aligned} W_{c,\text{can}} &= [B_c \quad A_c B_c \quad \dots \quad A_c^{n-1} B_c] \\ &= \begin{bmatrix} 1 & -a_1 & a_1^2 - a_2 & \dots & * \\ 0 & 1 & -a_1 & \dots & * \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & -a_1 \\ 0 & 0 & \dots & 0 & 1 \end{bmatrix}. \end{aligned}$$

The entries indicated by $*$ are expressions in the coefficients $a_1, \dots, a_n$ whose precise values are not relevant here. The matrix $W_{c,\text{can}}$ is upper triangular with ones on its diagonal, so $\det(W_{c,\text{can}}) = 1$. The pair (A_c, B_c) is therefore controllable for every choice of the coefficients, which explains the name *controllable canonical form*.

Remark 5.1. Canonical-form conventions differ between sources and textbook — they are related by a permutation of the state coordinates.

5.2 Transforming a system into controller canonical form

We now start from a controllable SISO state-space realization (A, B, C, D) and seek to transform this into a controller canonical form (A_c, B_c, C_c, D_c)

as introduced above. Under the convention above, $D = 0$, and the goal is to determine a change of state coordinates that transforms the given realization into controller canonical form.

We seek an invertible matrix T such that, under the state transformation $z = Tx$,

$$(A, B, C, 0) \mapsto (TAT^{-1}, TB, CT^{-1}, 0) = (A_c, B_c, C_c, 0).$$

First compute the characteristic polynomial of the given matrix A ,

$$\det(qI - A) = q^n + a_1q^{n-1} + \cdots + a_n.$$

These coefficients determine the desired matrices A_c and B_c above. It remains to determine the coordinate transformation T that produces them.

Let $W_c = [B \ AB \ A^2B \ \cdots \ A^{n-1}B]$ be the controllability matrix of the original realization. Under the coordinate transformation $z = Tx$, the transformed controllability matrix is

$$\begin{aligned} W_{c,\text{can}} &= [TB \ (TAT^{-1})TB \ (TAT^{-1})^2TB \ \cdots] \\ &= [TB \ TAB \ TA^2B \ \cdots] = TW_c. \end{aligned}$$

Hence

$$\boxed{T = W_{c,\text{can}}W_c^{-1}.}$$

(The inverse of W_c exists because the pair (A, B) is controllable.)

The equality $W_{c,\text{can}} = TW_c$ maps the controllability basis of the original system to that of the canonical system. Furthermore, we have

$$A_c = TAT^{-1}, \quad B_c = TB, \quad C_c = CT^{-1}, \quad D_c = 0.$$

Remark 5.2. It is sometimes more convenient to compute the inverse transformation T^{-1} . The inverse of the canonical controllability matrix has the particularly simple form

$$W_{c,\text{can}}^{-1} = \begin{bmatrix} 1 & a_1 & a_2 & \cdots & a_{n-1} \\ 0 & 1 & a_1 & \cdots & a_{n-2} \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & a_1 \\ 0 & 0 & \cdots & 0 & 1 \end{bmatrix},$$

so that

$$T^{-1} = W_cW_{c,\text{can}}^{-1}.$$

Example

Consider the realization

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad C = \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad D = 0.$$

Its characteristic polynomial is $p(q) = (q+1)(q+2) = q^2 + 3q + 2$. Therefore,

$$A_c = \begin{bmatrix} -3 & -2 \\ 1 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

The two controllability matrices are

$$W_c = \begin{bmatrix} 1 & -1 \\ 1 & -2 \end{bmatrix}, \quad W_{c,\text{can}} = \begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix}.$$

Consequently,

$$T = W_{c,\text{can}} W_c^{-1} = \begin{bmatrix} -1 & 2 \\ 1 & -1 \end{bmatrix}, \quad T^{-1} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}.$$

Substitution gives

$$TAT^{-1} = \begin{bmatrix} -3 & -2 \\ 1 & 0 \end{bmatrix}, \quad TB = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad CT^{-1} = \begin{bmatrix} 1 & 3 \end{bmatrix}.$$

Thus the transformed realization is in controller canonical form and has the same transfer function

$$H(q) = \frac{q+3}{q^2+3q+2}.$$

In a later chapter, the controller canonical form will be used to make state-feedback pole placement particularly transparent.

5.3 Other canonical forms

Other choices of coordinates lead to other useful standard forms. We state two important forms without developing their transformations in detail.

Observable (or observer) canonical form. For the strictly proper SISO transfer function above, an observable canonical realization is obtained from the controller canonical realization by duality:

$$A_o = A_c^T, \quad B_o = C_c^T, \quad C_o = B_c^T, \quad D_o = 0.$$

Explicitly,

$$A_o = \begin{bmatrix} -a_1 & 1 & 0 & \cdots & 0 \\ -a_2 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -a_{n-1} & 0 & 0 & \cdots & 1 \\ -a_n & 0 & 0 & \cdots & 0 \end{bmatrix}, \quad B_o = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix},$$

$$C_o = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}, \quad D_o = 0.$$

This form is useful in observer design. An observable realization can be transformed into this form using its observability matrix, analogously to the controllability construction above; we do not develop that construction here.

Diagonal or modal form. If A has a complete set of eigenvectors, write $A = V\Lambda V^{-1}$, where Λ contains the eigenvalues on its diagonal. The transformation $z = V^{-1}x$ gives

$$A_m = \Lambda, \quad B_m = V^{-1}B, \quad C_m = CV, \quad D_m = 0.$$

In these coordinates, the state dynamics are separated into independent modes, which can be related to the eigenvectors of A .

Remark 5.3. For an n -th order system described by real-valued state-space matrices, in general there are n complex eigenvalues. In such cases, the complex eigenvalues appear in conjugate pairs. In such cases, a block-diagonal modal form can be considered where the complex conjugate modes can be combined into real 2×2 blocks. This can be done by realizing that the following 2×2 matrix has $a + jb$ and $a - jb$ as its eigenvalues:

$$A = \begin{bmatrix} a & b \\ -b & a \end{bmatrix}$$

Remark 5.4. Not every $n \times n$ matrix is ‘fully’ diagonalizable; a matrix may not have a complete set of n eigenvectors. Such matrices can be represented using Jordan form, which contains triangular blocks associated with ‘generalized eigenvectors’.

Further reading

- M. Arcak and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

See Chapter 1, Sections B.5–B.7.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

See Sections 7.1 and 8.1.

6 Controllability, state feedback and eigenvalue placement

6.1 Controllability

A fundamental question in control is whether, and how, we can drive a linear system to the zero state – once we have established the methodology to do this, driving a system to another desired state becomes an easier task. Answering this question involves a bit of linear algebra and leads to some interesting and powerful results.

For convenience, we will derive the results for discrete-time systems, where the steps enable a straightforward interpretation. Note that similar conditions (resulting in the same matrices and rank conditions) can be obtained for continuous-time systems.

Consider the discrete-time system

$$x[k+1] = Ax[k] + Bu[k], \quad y[k] = Cx[k], \quad x[0] = x_0.$$

Successive substitution over values of $k = 0, 1, 2, \dots$ gives

$$\begin{aligned} x[1] &= Ax_0 + Bu[0], \\ x[2] &= A^2x_0 + ABu[0] + Bu[1], \quad \dots \\ x[M] &= A^Mx_0 + \underbrace{\begin{bmatrix} B & AB & \dots & A^{M-1}B \end{bmatrix}}_{W_c^{(M)}} \underbrace{\begin{bmatrix} u[M-1] \\ u[M-2] \\ \vdots \\ u[0] \end{bmatrix}}_U. \end{aligned}$$

The question if we can solve $0 = x[M] = A^Mx_0 + W_c^{(M)}U$ is determined by the rank of $W_c^{(M)}$. The columns of $W_c^{(M)}$ describe the state directions that the input sequence can produce at step M . A solution exists exactly when $-A^Mx_0$ belongs to this span.

We have to determine up to what value of M we need to go so that the matrix $W_c^{(M)}$ will not anymore increase in rank by increasing M . By the theorem

of Cayley-Hamilton, a matrix satisfies its own characteristic equation, so for an $n \times n$ matrix A , the power A^n can be written as a linear combination of lower powers $I, A, \dots, A^{n-1}$. Post-multiplying by B shows that $A^n B$ adds no new column directions. The same obviously holds for every higher power.

We define the *controllability matrix* W_c as

$$W_c = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix}.$$

Thus full controllability is equivalent to $\text{rank } W_c = n$.

Remark 6.1. With several inputs ($m > 1$), full rank may already be attained with fewer block columns.

To reach an arbitrary $x_f = x[M]$, one would solve $W_c^{(M)} U = x_f - A^M x_0$, which simply replaces the right-hand side of the equation by another vector. The interpretation is the same: If the columns span the full n -dimensional (state) space $\mathbb{R}^n$, then we can drive the system from $x_0 \in \mathbb{R}^n$ to any $x_f \in \mathbb{R}^n$.

6.2 State feedback and pole placement

We have established when we can steer a system from any initial state to any desired target state, and have derived the linear algebraic relations that govern this control problem. However, this viewpoint is not immediately useful, as it seems to require the solution of a system of linear equations in each time step k .

Instead, we will use the idea of ‘state feedback’, where we feed back the ‘measured’ full state to the input via a gain matrix $K \in \mathbb{R}^{m \times n}$ as in $u[k] = -Kx[k]$.

Apart from the intuition that negative feedback has the potential to reduce the state variable (to zero), it turns out that state feedback $u = -Kx$ allows us to achieve the intended control, where through the design of the feedback matrix K , we will be able to tune the (closed-loop) dynamics of the system under feedback.

The following statements are equivalent:

- $\text{rank } W_c = n$.
- Every target state $x_f \in \mathbb{R}^n$ can be reached from every initial state $x_0 \in \mathbb{R}^n$ in finitely many steps.
- State feedback $u[k] = -Kx[k]$ permits arbitrary eigenvalue placement (of the feedback system).

With state feedback, the closed-loop dynamics become

$$u[k] = -Kx[k] \implies x[k+1] = (A - BK)x[k].$$

If (A, B) is controllable, a suitable feedback matrix K allows us to assign the eigenvalues of $A - BK$ arbitrarily. For real-valued K , nonreal desired eigenvalues must occur in complex-conjugate pairs. Asymptotic stability requires that

$$|\lambda_i| < 1,$$

for every eigenvalue λ_i of $A - BK$.

For simple examples, the design of K can be done by matching the coefficients of the characteristic polynomials of $A - BK$ with the polynomial built from the desired eigenvalues. For more complicated cases, the structure of the controller canonical form is helpful. We will illustrate both approaches in the following sections.

6.3 Example: direct coefficient matching

$$A = \begin{bmatrix} 2 & 1 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Open-loop stability.

$$\det(\lambda I - A) = (\lambda - 2)(\lambda + 1) = \lambda^2 - \lambda - 2.$$

Eigenvalues 2, -1 : unstable since $|2| > 1$. Both eigenvalues are nonzero, so A is invertible. The mode at -1 alternates sign without decaying.

Controllability.

$$W_c = \begin{bmatrix} B & AB \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 2 & -2 \end{bmatrix}, \quad \det W_c = -10 \neq 0.$$

Desired eigenvalues: $1/2, 1/3$. With $K = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$,

$$A - BK = \begin{bmatrix} 2 - k_1 & 1 - k_2 \\ -2k_1 & -1 - 2k_2 \end{bmatrix}.$$

$$\begin{aligned} \det(\lambda I - (A - BK)) &= \lambda^2 + (k_1 + 2k_2 - 1)\lambda + 3k_1 - 4k_2 - 2, \\ (\lambda - 1/2)(\lambda - 1/3) &= \lambda^2 - 5/6\lambda + 1/6. \end{aligned}$$

Matching coefficients:

$$k_1 + 2k_2 = 1/6, \quad 3k_1 - 4k_2 = 13/6 \quad \implies \quad K = \begin{bmatrix} 1/2 & -1/6 \end{bmatrix}.$$

Thus

$$F := A - BK = \begin{bmatrix} 3/2 & 7/6 \\ -1 & -2/3 \end{bmatrix}.$$

Both eigenvalues lie inside the unit circle; $F^k \rightarrow 0$. Notice that the individual matrix entries need not lie between -1 and 1 : stability is determined by the eigenvalues, not by the entries. With zero reference, $x[k] = F^k x_0 \rightarrow 0$.

6.4 Feedback in controller canonical coordinates

For a controllable single-input system, write the characteristic polynomial as

$$p_A(\lambda) = \det(\lambda I - A) = \lambda^n + a_1 \lambda^{n-1} + a_2 \lambda^{n-2} + \cdots + a_n.$$

Using the controller canonical coordinates $z = Tx$ introduced earlier gives

$$A_c = TAT^{-1} = \begin{bmatrix} -a_1 & -a_2 & \cdots & -a_n \\ 1 & 0 & \cdots & 0 \\ 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 1 & 0 \end{bmatrix}, \quad B_c = TB = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

As throughout these notes, $D = D_c = 0$. The transformed output matrix is $C_c = CT^{-1}$, but neither C_c nor D_c enters the pole-placement calculation. The construction of this transformation has already been discussed; here we use the resulting structure to design the feedback gain.

Let $u[k] = -K_c z[k]$, with $K_c = [k_1 \ k_2 \ \cdots \ k_n]$. Since $B_c = e_1$, feedback changes only the first row:

$$A_c - B_c K_c = \begin{bmatrix} -(a_1 + k_1) & -(a_2 + k_2) & \cdots & -(a_n + k_n) \\ 1 & 0 & \cdots & 0 \\ 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 1 & 0 \end{bmatrix}.$$

Its characteristic polynomial is

$$p_{cl}(\lambda) = \lambda^n + (a_1 + k_1)\lambda^{n-1} + (a_2 + k_2)\lambda^{n-2} + \cdots + (a_n + k_n).$$

For two states, explicitly,

$$\det \begin{bmatrix} \lambda + a_1 + k_1 & a_2 + k_2 \\ -1 & \lambda \end{bmatrix} = \lambda^2 + (a_1 + k_1)\lambda + (a_2 + k_2).$$

Write the desired polynomial as

$$p_{\text{des}}(\lambda) = \prod_{j=1}^n (\lambda - \lambda_j) = \lambda^n + \alpha_1 \lambda^{n-1} + \alpha_2 \lambda^{n-2} + \cdots + \alpha_n.$$

Matching coefficients gives

$$k_i = \alpha_i - a_i, \quad K_c = \begin{bmatrix} \alpha_1 - a_1 & \alpha_2 - a_2 & \cdots & \alpha_n - a_n \end{bmatrix}.$$

Remark 6.2. Each gain adjusts one coefficient, not one eigenvalue independently; the eigenvalues are determined by the roots of the characteristic polynomial.

Return to the original coordinates. Because $z = Tx$,

$$u[k] = -K_c z[k] = -K_c T x[k] \implies K = K_c T.$$

Indeed,

$$A_c - B_c K_c = T(A - BK)T^{-1}.$$

6.5 Constant reference tracking

Assume a single input, a single output $y[k] = Cx[k]$, and a stable closed loop. Use

$$u[k] = -Kx[k] + k_r r, \quad x[k+1] = (A - BK)x[k] + Bk_r r.$$

Feedback $-Kx[k]$ stabilizes the origin. The constant term $k_r r$ shifts the equilibrium; choose its size so that the equilibrium output equals r . Recall that in discrete time equilibrium means the state repeats at the next step (not that the right-hand side is zero). Thus, for constant r , $x[k+1] = x[k] = x_{\text{ss}}$:

$$\begin{aligned} x_{\text{ss}} &= (A - BK)x_{\text{ss}} + Bk_r r, \\ (I - A + BK)x_{\text{ss}} &= Bk_r r, \\ x_{\text{ss}} &= (I - A + BK)^{-1} Bk_r r, \\ y_{\text{ss}} &= C(I - A + BK)^{-1} Bk_r r. \end{aligned}$$

Require $y_{ss} = r$. For $r \neq 0$, divide by r :

$$1 = C(I - A + BK)^{-1} B k_r \implies k_r = \frac{1}{C(I - A + BK)^{-1} B}.$$

Stability ensures $I - A + BK$ is invertible. The scalar denominator must be nonzero. The same k_r works for any constant reference; it compensates for the steady-state gain. This guarantees exact tracking only for the nominal model. Unlike integral action, it does not generally eliminate steady-state errors caused by constant disturbances or model mismatch.

To see convergence, subtract the equilibrium equation from the dynamics:

$$x[k+1] - x_{ss} = (A - BK)(x[k] - x_{ss}).$$

The chosen eigenvalues ensure $x[k] \rightarrow x_{ss}$, hence $y[k] \rightarrow r$.

Further reading

- M. Arcaç and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality* (2026).

See Chapter 5, especially Section A.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, second edition (2021).

See Chapter 7, especially Sections 7.1–7.3.

7 Observability and full state observers

7.1 Can we estimate the state?

Consider the discrete-time system

$$x[k+1] = Ax[k] + Bu[k], \quad y[k] = Cx[k].$$

State feedback $u[k] = -Kx[k]$ assumes that the entire state is available. Usually only the output $y[k]$ is measured. An observer uses the known inputs and measured outputs to estimate the state.

7.2 Observability as a linear system

We now address the question of determining the initial condition $x[0] = x_0$ from given (input and) output measurements. Assume the system is not actuated: $u[k] = 0$. Successive measurements satisfy

$$y[0] = Cx_0, \quad y[1] = CAx_0, \quad y[2] = CA^2x_0, \quad \dots$$

Finding x_0 can be done by stacking these equations into a linear system

$$\begin{bmatrix} y[0] \\ y[1] \\ \vdots \\ y[n-1] \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}}_{W_o} x_0.$$

The initial state x_0 is uniquely determined iff

$$\text{rank } W_o = n.$$

This is full column rank: distinct states must produce distinct measurement sequences. By the Cayley-Hamilton theorem, block rows involving higher powers of A cannot increase the rank once the first n block rows have been included. It is therefore sufficient to use W_o as defined above; no extended observability matrix is needed.

If the system is actuated (u is not a zero sequence), the known inputs do not change the rank result. We can subtract the contributions due to the input using the general expression

$$y[j] = CA^j x_0 + \sum_{i=0}^{j-1} CA^{j-1-i} Bu[i].$$

After reconstructing the initial state x_0 , the known dynamics and inputs determine later states as well.

Interpretation for state feedback control From the observations above, we conclude that the state components need not be measured directly: its influence may appear in later outputs, and so it can be reconstructed from the past inputs and outputs.

The rank condition on W_o can be understood as follows: if $W_o v = 0$ for a nonzero v , then the initial states x_0 and $x_0 + v$ would produce identical

outputs under the same input sequence. In such a case, no estimator can distinguish them from those data alone.

Notice that the algebraic expressions that are obtained are dual to those of controllability:

$$W_o(A, C) = W_c(A^\top, C^\top)^\top.$$

7.3 A running state observer

The preceding construction shows that, for an observable system, the state can be reconstructed from a sufficiently long sequence of known inputs and measured outputs.

For online use, rather than repeatedly performing this reconstruction, we propagate an estimate $\hat{x}$ and correct it using the output mismatch. We use the following predictor-form observer:

$$\hat{x}[k+1] = A\hat{x}[k] + Bu[k] + L(y[k] - C\hat{x}[k]).$$

The first two terms predict the next state from the model. The *innovation* $y[k] - C\hat{x}[k]$ compares the measurement with the predicted output. The gain L determines how this mismatch changes the next estimate.

Let us define $e[k] = x[k] - \hat{x}[k]$. With an exact model and no noise,

$$\begin{aligned} e[k+1] &= Ax[k] + Bu[k] - A\hat{x}[k] - Bu[k] - L(Cx[k] - C\hat{x}[k]) \\ &= (A - LC)e[k]. \end{aligned}$$

This means that $e[k] \rightarrow 0$ for every initial estimation error if all eigenvalues of $A - LC$ lie inside the unit circle.

7.4 Observer pole placement

If (A, C) is observable, the eigenvalues of $A - LC$ can be placed arbitrarily. If A and C are real and a real-valued observer gain L is required, nonreal desired poles must occur in complex-conjugate pairs. Practically, we simply use the methods for eigenvalue placement for state feedback, but on the transposed system:

$$(A - LC)^\top = A^\top - C^\top L^\top.$$

Remark 7.1. Full observability is needed for arbitrary placement. For convergence alone, *detectability* suffices: all unobservable modes must already lie strictly inside the unit circle. Such modes cannot be corrected from the measurements, but their errors decay without correction.

7.5 Example: coefficient matching

Reuse the controller example:

$$A = \begin{bmatrix} 2 & 1 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Only $x_1[k]$ is measured. Nevertheless,

$$W_o = \begin{bmatrix} C \\ CA \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad \det(W_o) = 1.$$

Indeed,

$$y[k+1] = 2y[k] + x_2[k] + u[k] \implies x_2[k] = y[k+1] - 2y[k] - u[k].$$

This explains observability in the noiseless case; it reconstructs $x_2[k]$ once the next measurement arrives. Direct differences also amplify measurement noise, motivating a running estimator.

Let $L = [\ell_1 \ \ell_2]^\top$. Then

$$A - LC = \begin{bmatrix} 2 - \ell_1 & 1 \\ -\ell_2 & -1 \end{bmatrix},$$

$$\det(\lambda I - (A - LC)) = \lambda^2 + (\ell_1 - 1)\lambda + \ell_1 + \ell_2 - 2.$$

Choose observer poles 0, 1/2. Comparing with $\lambda(\lambda - 1/2) = \lambda^2 - 1/2\lambda$ gives

$$\ell_1 - 1 = -1/2, \quad \ell_1 + \ell_2 - 2 = 0 \implies L = \begin{bmatrix} 1/2 \\ 3/2 \end{bmatrix}.$$

The zero pole is intentional: the corresponding error mode vanishes after one step. The other mode decays by a factor 1/2 per step.

$A = [2 \ 1; \ 0 \ -1]; B = [1; \ 2]; C = [1 \ 0];$

$L = \text{place}(A', C', [0 \ 1/2])';$

7.6 Estimated-state feedback and the Separation Principle

Now that we have a state observer, we can use the observer estimate in the state feedback controller.

$$u[k] = -K\hat{x}[k] + k_r r.$$

First take $r = 0$ and use the predictor observer introduced before. Since $\hat{x}[k] = x[k] - e[k]$,

$$x[k+1] = (A - BK)x[k] + BKe[k], \quad e[k+1] = (A - LC)e[k].$$

Therefore

$$\begin{bmatrix} x[k+1] \\ e[k+1] \end{bmatrix} = \begin{bmatrix} A - BK & BK \\ 0 & A - LC \end{bmatrix} \begin{bmatrix} x[k] \\ e[k] \end{bmatrix}.$$

This matrix is block triangular. Its eigenvalues are the union of those of $A - BK$ and $A - LC$. Thus controller and observer poles can be designed independently: if both sets lie inside the unit circle, the combined noise-free system is asymptotically stable. This is the *separation principle*.

For a constant reference, let $\tilde{x}[k] = x[k] - x_{ss}$ and retain $e[k] = x[k] - \hat{x}[k]$. The same block matrix describes the dynamics of $\tilde{x}$ and e . The previously derived reference scaling therefore remains applicable.

For the running example,

$$K = \begin{bmatrix} 1/2 & -1/6 \end{bmatrix}, \quad L = \begin{bmatrix} 1/2 \\ 3/2 \end{bmatrix}, \quad k_r = 1/12.$$

The combined poles are $1/2, 1/3, 0, 1/2$, all inside the unit circle. Separation concerns nominal stability; measurement noise still affects both the estimates and the control input.

For practical purposes, the estimation error is usually designed to decay faster than the controlled response, so that the observer transient does not dominate the closed-loop transient. As a rough starting point, one may choose observer poles that achieve in one step the error reduction that the controller poles achieve in two or three steps. For example, a dominant controller pole with magnitude 0.8 suggests observer-pole magnitudes around $0.8^2 = 0.64$ or $0.8^3 \approx 0.51$. Observer poles should not, however, be placed arbitrarily close to the origin: faster observers can require larger gains and increase sensitivity to measurement noise and modeling errors.

Further reading

- M. Arcaç and K. Poolla, *Linear Control Systems: Structure, Stability, Optimality*, 2026.

See Chapter 6, Sections A and B.

- K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed., 2021.

See Chapter 8, Sections 8.1–8.3.

8 Optimal Control

8.1 Controller performance

Consider the discrete-time linear system

$$x[k+1] = Ax[k] + Bu[k],$$

in which $x[k] \in \mathbb{R}^n$ is the state and $u[k] \in \mathbb{R}^m$ is the control input at time k . We have previously considered state feedback of the form

$$u[k] = -Kx[k],$$

for which the resulting closed-loop system is

$$x[k+1] = (A - BK)x[k].$$

If the system is controllable, eigenvalue placement allows us to choose an appropriate matrix K such that the eigenvalues of $A - BK$ are at desired locations inside the unit circle.

Depending on which eigenvalues we choose, the resulting state and control trajectories vary. If we have two controllers that stabilize the system, we need a way to determine which one is better. For example, whereas one control law might drive the state to zero rapidly while requiring too much energy, another might require a longer time but spend significantly less energy. To compare these two control laws and pick one that suits best for our given problem, we need a way to measure their performance.

8.2 Finite-horizon optimal control

Consider a system

$$x[k+1] = f(x[k], u[k]).$$

For example for our linear system $f(x[k], u[k]) = Ax[k] + Bu[k]$. Suppose the initial condition $x[0]$ is known and we consider a finite number of steps $k = 0, 1, \dots, N-1$ that the control is applied.

A general finite-horizon optimal control has the form

$$\begin{aligned} \min_{u[0], \dots, u[N-1]} \quad & J(u[0], \dots, u[N-1]; x[0]) \\ \text{s.t.} \quad & x[k+1] = f(x[k], u[k]), \quad k = 0, \dots, N-1, \\ & x[0] \text{ given.} \end{aligned} \quad (1)$$

The objective (cost) function is typically written as

$$J(u[0], \dots, u[N-1]; x[0]) = \ell_F(x[N]) + \sum_{k=0}^{N-1} \ell(x[k], u[k]), \quad (2)$$

in which $\ell_F(x[N])$ is called the **terminal cost** which associates the terminal state with a nonnegative cost and $\ell(x[k], u[k])$ is called the **running cost** which associates a state and control pair a nonnegative cost. The terminal cost penalizes undesirable final state. The running cost is a stage-additive cost, meaning that it only depends on the current state and control, and penalizes the behavior along the state and control trajectories. The initial state $x[0]$ inside the objective function is a parameter, and not an optimization variable because the cost associated with the same sequence of control inputs may vary when applied from different initial states.

In certain cases, there might be additional constraints on the values that the state and the control can take. For example, our control input might be bounded, or we may wish the state to remain within a given set. We will consider these problems later.

8.2.1 Example: Optimal control problem

Consider a one-dimensional linear system

$$x[k+1] = ax[k] + bu[k],$$

in which x and u are scalars. Suppose we want to minimize (i) the deviation from the origin $x = 0$ and (ii) the control energy spent for this purpose. Then, the objective function is given by

$$J(u[0], \dots, u[N-1]; x[0]) = (x[N])^2 + \sum_{k=0}^{N-1} q(x[k])^2 + r(u[k])^2,$$

in which $q > 0$ and $r > 0$ are the weights that determine the trade-off between the two objectives. Often, the state and control are additionally constrained to be in respective set of admissible values. We will not consider such problems in detail in this chapter.

8.2.2 Dynamic Programming

A finite-horizon optimal control problem can, in principle, be solved by optimizing directly over the entire control sequence $(u[0], \dots, u[N-1])$. For a given initial state x_0 , the corresponding state trajectory is determined by the system dynamics, so the problem may be viewed as an optimization over all control inputs simultaneously. This direct approach can be effective, and it will be useful later when we introduce *model predictive control*. However, it has two limitations. First, the number of optimization variables grows linearly with the prediction horizon, since all control inputs must be optimized at once. Second, solving the problem for a particular x_0 generally provides an optimal control sequence only for that initial condition. If the state changes, for example under disturbances that were not considered when obtaining a solution, the optimization problem must be solved again.

Unlike direct optimization, dynamic programming exploits the sequential structure of the problem. This is the viewpoint that we will take in this section and in the subsequent sections. Dynamic programming relies on a simple principle called the *principle of optimality* which, informally, states that the remaining tail of an optimal control sequence is optimal for the subproblem starting from the state reached at that time. By solving these subproblems recursively backward in time, dynamic programming determines the optimal cost-to-go (the cost of applying optimal controls from the current state onward), consequently, the optimal control as a function of the current state. Therefore, rather than computing a single sequence of optimal controls for a given initial state, dynamic programming naturally leads to an optimal state-feedback policy.

Principle of Optimality Let $(u^*[0], \dots, u^*[N-1])$ be an optimal control trajectory and let $(x^*[0], \dots, x^*[N])$ be the corresponding optimal state trajectory starting at $x^*[0] = x[0] = x_0$. Consider a tail subproblem defined as in the following. Starting from state $x^*[k]$, minimize the cumulative cost

$$J(u[k], \dots, u[N-1]; x^*[k]) := \ell_F(x[N]) + \sum_{i=k}^{N-1} \ell(x[i], u[i])$$

over all sequences $(u[k], \dots, u[N-1])$ subject to $x[k] = x^*[k]$ and $x[i+1] = f(x[i], u[i])$ for $i = k, \dots, N-1$. Then, the tail of the optimal sequence $(u^*[k], \dots, u^*[N-1])$ is optimal for the tail subproblem. This is called the principle of optimality.

We can use principle of optimality to determine a single optimal control at a time by solving longer tail problems using the result of the shorter ones.

Cost-to-go function Let $G_k^* : X \rightarrow \mathbb{R}_{\geq 0}$, be a function over the state space X that associates with each state the cost that will incur from following an optimal sequence of controls from time k on. G_k^* is called the optimal cost-to-go function³ at time k .

At time $k = 0$ we have

$$G_0^*(x_0) = \min_{u[0], \dots, u[N-1]} \{ \ell_F(x[N]) + \sum_{k=0, x[0]=x_0}^{N-1} \ell(x[k], u[k]) \},$$

with $x[k+1] = f(x[k], u[k])$ for $k = 0, \dots, N-1$. Applying control $u[k]$ at state $x[k]$ at time k takes us to the next state $x[k+1]$ at time $k+1$. The minimization on the right-hand side corresponds to the original problem we are trying to solve.

At time $k = N$ do not need to pick any control, so we have

$$G_N^*(x[N]) = \ell_F(x[N]).$$

At some time k , $0 < k < N$ we have

$$G_k^*(x[k]) = \min_{u[k], \dots, u[N-1]} \{ \ell_F(x[N]) + \sum_{i=k}^{N-1} \ell(x[i], u[i]) \}$$

We can rewrite this as

$$G_k^*(x[k]) = \min_{u[k]} \{ \ell(x[k], u[k]) + \underbrace{\min_{u[k+1], \dots, u[N-1]} \{ \ell_F(x[N]) + \sum_{i=k+1}^{N-1} \ell(x[i], u[i]) \}}_{G_{k+1}^*(x[k+1])} \}$$

This relationship gives **Bellman's Equation (recurrence equation)**:

$$\boxed{\begin{aligned} G_k^*(x[k]) &= \min_{u[k]} \{ \ell(x[k], u[k]) + G_{k+1}^*(x[k+1]) \} \\ &= \min_{u[k]} \{ \ell(x[k], u[k]) + G_{k+1}^*(f(x[k], u[k])) \}. \end{aligned}} \quad (3)$$

Bellman's equation leads to the **Dynamic Programming Algorithm**:

1. For $k = N$, define

$$G_N^*(x) := \ell_F(x), \text{ for all } x \in X.$$

³It is also called a value function. However, the notion of value is more suitable for reward maximization such as its use in reinforcement learning.

2. For $k \in \{0, \dots, N-1\}$, define recursively (proceeding backwards from $k = N-1$)

$$G_k^*(x) := \min_u \{ \ell(x, u) + G_{k+1}^*(f(x, u)) \}, \text{ for all } x \in X.$$

To construct an optimal control sequence for an initial state $x[0] = x_0$:

1. Set

$$u^*[0] \in \operatorname{argmin}_u \{ \ell(x[0], u) + G_1^*(x[1]) \} \quad \text{and} \quad x[1] := f(x[0], u).$$

2. Going forward in time, for $k = \{1, \dots, N-1\}$ set

$$u^*[k] \in \operatorname{argmin}_u \{ \ell(x^*[k], u) + G_{k+1}^*(f(x^*[k], u)) \} \quad \text{and} \quad x^*[k+1] := f(x^*[k], u^*[k]).$$

8.2.3 Example: Finite state/control spaces DP

When the number of states and controls are finite we can directly apply the dynamic programming algorithm by enumerating all possible controls for each state at each time step.

Consider the finite state space and finite action space

$$X = \{0, 1, 2, 3\}, \quad U = \{-1, 0, 1\},$$

and the dynamics

$$x[k+1] = f(x[k], u[k]) := \min\{3, \max\{0, x[k] + u[k]\}\}.$$

Thus, the state is moved by the input and then saturated to remain in the set X .

Let the horizon be $N = 3$, the stage cost be

$$\ell(x[k], u[k]) = x[k]^2 + u[k]^2,$$

and the terminal cost be

$$\ell_F(x[N]) = 4x[N]^2.$$

The Bellman recursion is

$$G_N^*(x) = \ell_F(x),$$

and for $k = N-1, \dots, 0$,

$$G_k^*(x) = \min_{u \in U} \{ \ell(x, u) + G_{k+1}^*(f(x, u)) \}.$$

Step 1: terminal cost. At time $k = N = 3$, we have

$$G_3^*(x) = 4x^2.$$

Hence

$$G_3^*(0) = 0, \quad G_3^*(1) = 4, \quad G_3^*(2) = 16, \quad G_3^*(3) = 36.$$

Step 2: one step backwards. Consider the state $x[2] = 3$. Then

$$G_2^*(3) = \min_{u \in \{-1, 0, 1\}} \{3^2 + u^2 + G_3^*(f(3, u))\}.$$

Evaluating the three controls gives

$$\begin{aligned} u[2] = -1 : \quad & 3^2 + (-1)^2 + G_3^*(2) = 9 + 1 + 16 = 26, \\ u[2] = 0 : \quad & 3^2 + 0^2 + G_3^*(3) = 9 + 0 + 36 = 45, \\ u[2] = 1 : \quad & 3^2 + 1^2 + G_3^*(3) = 9 + 1 + 36 = 46. \end{aligned}$$

Therefore,

$$G_2^*(3) = 26, \quad u_2^*(3) = -1.$$

In the same way, one obtains for all states:

x	0	1	2	3
$G_3^*(x)$	0	4	16	36
$G_2^*(x)$	0	2	9	26
$u_2^*(x)$	0	-1	-1	-1

Step 3: another step backwards. Using the values of G_2^* , we compute for $x[1] = 3$:

$$G_1^*(3) = \min_{u \in \{-1, 0, 1\}} \{3^2 + u^2 + G_2^*(f(3, u))\}.$$

Again evaluating all inputs:

$$\begin{aligned} u[1] = -1 : \quad & 3^2 + (-1)^2 + G_2^*(2) = 9 + 1 + 9 = 19, \\ u[1] = 0 : \quad & 3^2 + 0^2 + G_2^*(3) = 9 + 0 + 26 = 35, \\ u[1] = 1 : \quad & 3^2 + 1^2 + G_2^*(3) = 9 + 1 + 26 = 36. \end{aligned}$$

Hence,

$$G_1^*(3) = 19, \quad u_1^*(3) = -1.$$

For all states at $k = 1$,

x	0	1	2	3
$G_1^*(x)$	0	2	7	19
$u_1^*(x)$	0	-1	-1	-1

The recursion can be continued to $k = 0$. In particular, for an initial state $x_0 = 3$, the optimal control at every stage is -1 .

This example illustrates how, for finite state and action spaces, dynamic programming reduces the original finite-horizon problem to a sequence of simple minimization problems evaluated backward in time.

Computational complexity The advantage of dynamic programming becomes apparent as the horizon increases. Suppose that the state and action spaces contain

$$|X| = n_x, \quad |U| = n_u$$

elements. If the problem is solved by direct enumeration for a fixed initial state, there are

$$n_u^N$$

possible control sequences of length N . Thus, the number of candidate sequences grows exponentially with the horizon. If the cost of each sequence is evaluated over the complete trajectory, the computational complexity is

$$\mathcal{O}(n_u^N).$$

Dynamic programming avoids repeatedly solving identical subproblems. At each time step, using the Bellman equation only the admissible controls for each state are evaluated. Assuming that the state transition and stage cost can be evaluated in constant time, its computational complexity is

$$\mathcal{O}(Nn_xn_u),$$

which grows linearly with the horizon for fixed finite state and action spaces.

8.3 Dynamic Programming in Continuous Spaces

Extending dynamic programming to continuous state and control spaces creates additional challenges. In the finite-state case, we could store the cost-to-go function as a table containing one value for every state. For a continuous state space, however, there are infinitely many possible states, so

the cost-to-go function cannot in general be evaluated and stored through enumeration.

One possible approach is to *discretize* the state and control spaces. That is, we replace the continuous spaces by finite grids and apply the same dynamic programming recursion as in the finite-state case. For example, if the state space is discretized using N_x grid points in each state dimension, then an d_x -dimensional state space contains approximately

$$N_x^{d_x}$$

grid points. Similarly, using N_u grid points in each of the d_u control dimensions results in approximately

$$N_u^{d_u}$$

possible control values.

Bellman's equation can then be evaluated approximately on these grids. However, the next state

$$x[k+1] = f(x[k], u[k])$$

will in general not coincide exactly with a grid point. The value $G_{k+1}^*(x[k+1])$ must therefore typically be approximated, for example by interpolating between neighboring grid points.

For a finite horizon of length N , a straightforward grid-based dynamic programming implementation requires approximately

$$\mathcal{O}(N N_x^{d_x} N_u^{d_u})$$

state-control evaluations. Thus, although dynamic programming avoids the exponential growth with the horizon associated with direct enumeration of complete control sequences, its computational requirements grow exponentially with the dimensions of the state and control spaces.

This phenomenon is known as the *curse of dimensionality*. Even a relatively coarse discretization can become impractical when the number of state variables is large. For example, using only 10 grid points per state dimension requires 10^{d_x} values to represent the cost-to-go function. Thus, 4 state variables require 10^4 grid points, whereas 10 state variables already require 10^{10} grid points. Increasing the grid resolution further increases the computational and memory requirements.

Consequently, direct discretization of continuous spaces is mainly practical for problems with relatively low-dimensional state and control spaces. For

higher-dimensional problems, other approaches are required, such as approximating the cost-to-go function (using for example basis functions, or neural networks) or exploiting special structure in the optimal control problem.

An important class of continuous-state and continuous-control problems for which the dynamic programming recursion can be solved analytically is when the system dynamics are linear and the cost is quadratic. Solution to linear-quadratic problems is also useful beyond systems that are linear and costs that are quadratic. A nonlinear system can be linearized around a desired state, and a nonlinear cost can be approximated locally by a quadratic function. The resulting controller then provides a local approximation to the solution of the original nonlinear optimal control problem. We will see this type of approximation in the following chapters.

8.4 Linear Quadratic Problems

8.4.1 Example: Scalar case

Consider a scalar discrete-time linear system given by

$$x[k+1] = ax[k] + bu[k].$$

We assume no bounds on either states or controls. To keep the algebra simple, we first consider the special case with a running cost that includes no state penalty, therefore, we wish to minimize the cost:

$$J(u[0], \dots, u[N-1]; x_0) = \frac{1}{2}h(x[N])^2 + \frac{1}{2} \sum_{k=0, x[0]=x_0}^{N-1} r(u[k])^2.$$

Above, $r > 0$ and $h > 0$ are weights that express the relative importance between the control effort $(u[k])^2$ and the deviation $(x[N] - x_f)^2$ between the final state $x[N]$ and the desired terminal state $x_f = 0$.

We now solve this problem for a fixed length of N stages. Recall the dynamic programming recurrence:

$$G_k^*(x) = \min_{u \in \mathbb{R}} \{ \ell(x, u) + G_{k+1}^*(f(x, u)) \}.$$

We can compute G_k^* for all $k = 0, \dots, N$ recursively, by starting from $k = N$ and proceeding backwards.

Time $k = N$

$$\begin{aligned} G_N^*(x[N]) &= \ell_F(x[N]) = \frac{1}{2}h(x[N])^2 \\ &= \frac{1}{2}S_N(x[N])^2 \quad (\text{where } S_N := h) \end{aligned} \tag{4}$$

Time $k = N - 1$

$$\begin{aligned}
G_{N-1}^*(x[N-1]) &= \min_{u[N-1] \in \mathbb{R}} \left\{ \frac{1}{2}r(u[N-1])^2 + G_N^*(x[N]) \right\} \\
&= \min_{u[N-1] \in \mathbb{R}} \left\{ \frac{1}{2}r(u[N-1])^2 + \frac{1}{2}S_N(x[N])^2 \right\} \quad (G_N^*(x[N]) = \frac{1}{2}S_N(x[N])^2) \\
&= \min_{u[N-1] \in \mathbb{R}} \left\{ \frac{1}{2}r(u[N-1])^2 + \frac{1}{2}S_N(ax[N-1] + bu[N-1])^2 \right\}
\end{aligned} \tag{5}$$

in which $x[N] := ax[N-1] + bu[N-1]$.

The optimal control $u^*[N-1]$ that minimizes the expression inside the curly brackets is obtained by setting its derivative with respect to $u[N-1]$ to zero, which implies

$$r(u^*[N-1]) + S_N b(ax[N-1] + bu^*[N-1]) = 0$$

Solving for $u^*[N-1]$ we obtain the optimal control at time $N-1$ as a function of $x[N-1]$ given by

$$u^*[N-1] = -\frac{S_N b a x[N-1]}{r + S_N b^2}. \tag{6}$$

Denoting, $K_{N-1} := \frac{S_N b a}{r + S_N b^2}$ we obtain the optimal control as linear state-feedback

$$u^*[N-1] = -K_{N-1}x[N-1]. \tag{7}$$

We can now rewrite the cost-to-go function at time $N-1$ in terms of S_{N-1} and K_{N-1} :

$$\begin{aligned}
G_{N-1}^*(x[N-1]) &= \frac{1}{2}r(u^*[N-1])^2 + \frac{1}{2}S_N(ax[N-1] + bu^*[N-1])^2 \\
&= \frac{1}{2}r(-K_{N-1}x[N-1])^2 + \frac{1}{2}S_N(ax[N-1] - bK_{N-1}x[N-1])^2 \\
&= \frac{1}{2}rK_{N-1}^2x[N-1]^2 + \frac{1}{2}S_N(a - bK_{N-1})^2x[N-1]^2 \\
&= \frac{1}{2}[rK_{N-1}^2 + S_N(a - bK_{N-1})^2]x[N-1]^2 \\
&= \frac{1}{2}S_{N-1}x[N-1]^2,
\end{aligned} \tag{8}$$

with

$$S_{N-1} := rK_{N-1}^2 + S_N(a - bK_{N-1})^2. \tag{9}$$

Time $k < N - 1$

We can now repeat the same argument for any earlier time k . Assume that the optimal cost-to-go at time $k + 1$ is of the form

$$G_{k+1}^*(x[k+1]) = \frac{1}{2}S_{k+1}(x[k+1])^2. \quad (10)$$

Then, using $x[k+1] = ax[k] + bu[k]$, the dynamic programming recurrence gives

$$G_k^*(x[k]) = \min_{u[k] \in \mathbb{R}} \left\{ \frac{1}{2}r(u[k])^2 + \frac{1}{2}S_{k+1}(ax[k] + bu[k])^2 \right\}. \quad (11)$$

The minimizing control is found by differentiating with respect to $u[k]$:

$$ru^*[k] + S_{k+1}b(ax[k] + bu^*[k]) = 0. \quad (12)$$

Therefore,

$$u^*[k] = -\frac{S_{k+1}ba}{r + S_{k+1}b^2}x[k]. \quad (13)$$

Defining

$$K_k := \frac{S_{k+1}ba}{r + S_{k+1}b^2}, \quad (14)$$

the optimal control at time k is

$$u^*[k] = -K_k x[k]. \quad (15)$$

Substituting this control into the cost-to-go gives

$$\begin{aligned} G_k^*(x[k]) &= \frac{1}{2}rK_k^2(x[k])^2 + \frac{1}{2}S_{k+1}(a - bK_k)^2(x[k])^2 \\ &= \frac{1}{2}[rK_k^2 + S_{k+1}(a - bK_k)^2](x[k])^2. \end{aligned} \quad (16)$$

Hence, defining

$$S_k := rK_k^2 + S_{k+1}(a - bK_k)^2, \quad (17)$$

we again obtain a quadratic cost-to-go:

$$G_k^*(x[k]) = \frac{1}{2}S_k(x[k])^2. \quad (18)$$

Using the expression for K_k , the recursion for S_k simplifies to

$$S_k = \frac{a^2rS_{k+1}}{r + b^2S_{k+1}}$$

with terminal condition

$$S_N = h.$$

Thus, starting from $S_N = h$, we can compute

$$S_{N-1}, S_{N-2}, \dots, S_0$$

backwards. At each time k , the corresponding optimal control is

$$\boxed{u^*[k] = -K_k x[k], \quad K_k = \frac{S_{k+1} b a}{r + b^2 S_{k+1}}.} \quad (19)$$

Finally, for the initial state $x[0] = x_0$, the minimum cost is

$$G_0^*(x_0) = \frac{1}{2} S_0 x_0^2. \quad (20)$$

8.4.2 Linear Quadratic Regulator

For the linear system dynamics considered in the previous example, the dynamic programming recursion can be solved analytically. In particular, we obtained an explicit expression for both the optimal cost-to-go function and the optimal control policy at every stage. The resulting optimal control is a linear function of the state, while the optimal cost-to-go is a quadratic function of the state. Therefore, in this special class of problems, no numerical approximation of the dynamic programming recursion is required. Optimal control problems with linear system dynamics and quadratic cost functions form the basis of the **Linear Quadratic Regulator (LQR)**. The term *regulation* refers to the objective of driving the state of the system toward the origin⁴.

We now extend the previous scalar example to the general n -dimensional case. Consider the discrete-time linear system

$$x[k+1] = Ax[k] + Bu[k], \quad (21)$$

in which $x[k] \in \mathbb{R}^n$, $u[k] \in \mathbb{R}^m$, and $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$. We wish to minimize the finite-horizon quadratic cost

$$J(u[0], \dots, u[N-1]; x_0) = \frac{1}{2} x[N]^T H x[N] + \frac{1}{2} \sum_{k=0, x[0]=x_0}^{N-1} (x[k]^T Q x[k] + u[k]^T R u[k]). \quad (22)$$

⁴A nonzero reference state can be handled by an appropriate change of variables.

Here, $H, Q \in \mathbb{R}^{n \times n}$ are symmetric positive semi-definite matrices, and $R \in \mathbb{R}^{m \times m}$ is symmetric positive definite. The term $x[k]^T Q x[k]$ penalizes deviations of the state from the desired state $x_f = 0$, while $u[k]^T R u[k]$ penalizes the control effort. The terminal term $x[N]^T H x[N]$ penalizes deviation from the desired state at the end of the horizon.

As in the scalar case, we assume that there are no additional constraints on the states or controls.

Positive definite and positive semi-definite matrices. A symmetric matrix $M \in \mathbb{R}^{n \times n}$ is called *positive definite*, written $M \succ 0$, if $z^T M z > 0$ for every nonzero vector $z \in \mathbb{R}^n$. It is called *positive semi-definite*, written $M \succeq 0$, if $z^T M z \geq 0$ for every vector $z \in \mathbb{R}^n$. Thus, $Q \succeq 0$ and $H \succeq 0$ ensure that state penalties are never negative, while $R \succ 0$ ensures that every nonzero control input has strictly positive cost.

For the finite-horizon LQR problem, the optimal control at time k is a linear state-feedback law:

$$u^*[k] = -K_k x[k], \quad (23)$$

$$K_k = (R + B^T S_{k+1} B)^{-1} B^T S_{k+1} A, \quad (24)$$

$$S_k = Q + K_k^T R K_k + (A - B K_k)^T S_{k+1} (A - B K_k), \quad (25)$$

$$G_k^*(x[k]) = \frac{1}{2} x[k]^T S_k x[k]. \quad (26)$$

Thus, as in the scalar case, the optimal control is linear in the state. Since the horizon is finite, the feedback gain K_k generally depends on time. Similarly, the optimal cost-to-go is quadratic in the state, with the matrix S_k also depending on time.

The matrices K_k and S_k are obtained recursively by working backwards from the terminal time.

Backward recursion

1. Set the terminal condition $S_N = H$.
2. For $k = N - 1, N - 2, \dots, 0$ compute

$$K_k = (R + B^T S_{k+1} B)^{-1} B^T S_{k+1} A, \quad (27)$$

followed by

$$S_k = Q + K_k^T R K_k + (A - B K_k)^T S_{k+1} (A - B K_k). \quad (28)$$

Once the backward recursion has been completed, the optimal control can be applied forward in time according to

$$u^*[k] = -K_k x[k], \quad k = 0, \dots, N-1. \quad (29)$$

Note that the matrices K_k and S_k depend only on the system matrices A and B , the cost matrices Q , R , and H , and the horizon length. They do not depend on the particular value of the state $x[k]$. Consequently, for a fixed system and a fixed horizon, the complete sequence

$$K_0, K_1, \dots, K_{N-1}$$

can be computed offline before the controller is applied.

8.5 Infinite-horizon optimal control

So far, we have considered optimal control problems over a fixed finite horizon N . We now investigate what happens as the horizon is extended indefinitely. For each finite value of N , dynamic programming gives a sequence of optimal cost-to-go functions and optimal feedback laws. If the system dynamics and the stage cost are time-invariant, then the same Bellman recursion is applied repeatedly as the horizon is extended. Under appropriate conditions, the influence of the terminal condition becomes progressively smaller for stages far from the terminal time, and the optimal cost-to-go functions and feedback gains converge to stationary values. This motivates the *infinite-horizon* optimal control problem, obtained by considering the limit

$$N \rightarrow \infty.$$

In this case, the controller is applied indefinitely and the cost is accumulated over

$$k = 0, 1, 2, \dots$$

An important issue arises when passing from a finite to an infinite horizon is that the accumulated cost must remain finite. Even if the cost incurred at every individual time step is finite, the sum of infinitely many such terms may diverge. For an infinite-horizon optimal control problem to have a meaningful solution, there must exist a controller that ensures that the cumulative cost is finite, that is, the sequence of running cost terms must tend to zero.

For regulation type of problems this typically means that the controller must drive the state and the control input to zero, that is $x[k] \rightarrow 0$ and $u[k] \rightarrow 0$ as

$k \rightarrow \infty$ at a rate that makes the infinite sum converge. If the state remains away from the desired equilibrium, or grows without bound, then a positive state penalty accumulated over infinitely many time steps will generally cause the total cost to diverge. Therefore, before considering the optimal infinite-horizon controller, we must establish that there exists at least one control law for which the accumulated cost is finite.

8.5.1 Infinite-horizon LQR

Consider the discrete-time linear system

$$x[k+1] = Ax[k] + Bu[k], \quad (30)$$

in which $x[k] \in \mathbb{R}^n$, $u[k] \in \mathbb{R}^m$, and $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$. We assume that the system is controllable. For a finite-horizon N consider the quadratic cost as

$$J_N(u[0], \dots, u[N-1]; x_0) = \frac{1}{2} \sum_{k=0, x[0]=x_0}^{N-1} (x[k]^T Q x[k] + u[k]^T R u[k]), \quad (31)$$

in which Q and R are symmetric positive definite weight matrices and the terminal cost is 0. We are interested in determining what happens as $N \rightarrow \infty$. The infinite-horizon cost is then defined as

$$J_\infty(\{u[k]\}_{k=0}^\infty; x_0) := \lim_{N \rightarrow \infty} J_N(u[0], \dots, u[N-1]; x_0). \quad (32)$$

Since all terms in the sum are nonnegative, the cumulative cost is nondecreasing as N increases. Therefore, to determine a control law that optimizes the given cost, we first need to establish that there exists at least a control law under which the given sum remains bounded (it does not tend to infinite) as $N \rightarrow \infty$.

Finite cost Because the system given by Equation 30 is assumed to be controllable, we can find controls $(u[0], u[1], \dots, u[M])$ for some M that drives an initial state x_0 to the origin, that is, after applying these controls the state at time $M+1$ becomes $x[M+1] = 0$. Then, for the rest of the horizon we can apply $u[k] = 0$ for $k > M$, meaning that the state will remain at the origin as well. The cost associated with this control law is therefore finite. Since an optimal control law cannot have a larger cost than any feasible control law, we have

$$J_\infty(u^*[0], \dots; x_0) \leq J_\infty(u[0], \dots, u[M], 0, \dots; x_0) < \infty, \quad (33)$$

in which $J_\infty(u^*[0], \dots; x_0)$ is the cost under an optimal control.

We now consider what happens to the finite-horizon LQR solution as the horizon N tends to infinity. Recall that, for a finite horizon, the optimal control and cost-to-go functions are given by

$$u^*[k] = -K_k x[k], \quad (34)$$

and

$$G_k^*(x[k]) = \frac{1}{2} x[k]^T S_k x[k], \quad (35)$$

in which the matrices K_k and S_k are computed backwards from the terminal time. For a finite horizon, these matrices generally depend on k . However, the system dynamics

$$x[k+1] = Ax[k] + Bu[k]$$

and the running cost

$$\frac{1}{2} (x[k]^T Q x[k] + u[k]^T R u[k])$$

are both time-invariant. As the horizon is extended, stages sufficiently far from the terminal time become indistinguishable from one another.

Under the assumptions that the system is controllable and $Q \succ 0, R \succ 0$ the finite-horizon recursion given by Bellman's equation converges to a stationary solution. In particular, as $N \rightarrow \infty$,

$$S_k^{(N)} \longrightarrow S, \quad K_k^{(N)} \longrightarrow K, \quad (36)$$

in which S and K are constant matrices. The superscript N denotes that $S_k^{(N)}$ and $K_k^{(N)}$ are the matrices computed for time k considering a horizon of length N . Consequently, the infinite-horizon optimal cost-to-go is time-invariant and has the form

$$G^*(x) = \frac{1}{2} x^T S x. \quad (37)$$

Note that the cost-to-go function is now independent of the time. Similarly, the optimal control law becomes the stationary state-feedback law

$$u^*[k] = -Kx[k]. \quad (38)$$

At the stationary solution, one additional step of the recursion must leave the matrix S unchanged. Thus, setting

$$S_k = S_{k+1} = S$$

in the recursion gives

$$K = (R + B^T S B)^{-1} B^T S A, \quad (39)$$

and

$$S = Q + A^T S A - A^T S B (R + B^T S B)^{-1} B^T S A. \quad (40)$$

Equation (40) is called the **Discrete-time Algebraic Riccati Equation (DARE)**. The DARE is a nonlinear matrix equation and, except for simple special cases, is generally not solved analytically. In practice, numerical algorithms are used to compute its stabilizing positive-definite solution S .

The infinite-horizon LQR controller is therefore

$$\boxed{u^*[k] = -Kx[k]} \quad (41)$$

with the same feedback gain K given by Equation (39) applied at every time step. The corresponding optimal cost from an initial state x_0 is

$$\boxed{G^*(x_0) = \frac{1}{2} x_0^T S x_0.} \quad (42)$$

Closed-loop stability An important consequence of the infinite-horizon LQR solution is that the optimal feedback controller

$$u^*[k] = -Kx[k]$$

makes the closed-loop system

$$x[k+1] = (A - BK)x[k]$$

asymptotically stable.

To see this, recall that the optimal infinite-horizon cost is finite:

$$J_\infty^*(x_0) := \frac{1}{2} \sum_{k=0}^{\infty} (x[k]^T Q x[k] + u^*[k]^T R u^*[k]) < \infty.$$

Since every term in the sum is nonnegative, finiteness of the total cost implies that

$$x[k]^T Q x[k] \rightarrow 0 \quad \text{as } k \rightarrow \infty.$$

Because $Q \succ 0$, the quantity $x^T Q x$ is zero only when $x = 0$, therefore,

$$x[k] \rightarrow 0 \quad \text{as } k \rightarrow \infty.$$

Hence, the optimal infinite-horizon LQR controller drives the state asymptotically to the origin. Equivalently, the eigenvalues of the closed-loop matrix $A - BK$ lie strictly inside the unit circle. Note that the state generally does not reach the origin in a finite number of steps, but it converges to the origin as $k \rightarrow \infty$.

Finally, we used the assumptions that the system is controllable, and $Q \succ 0$ and $R \succ 0$. These are sufficient conditions, however, they are not necessary. In particular, controllability ensures that there exists a stabilizing controller, and $Q \succ 0$ and $R \succ 0$ ensure that the deviation from 0 for each component is penalized. The existence of infinite-horizon LQR (following the same bounded cumulative cost argument) is also guaranteed under the following weaker assumptions: the system is stabilizable, the pair $(A, Q^{1/2})$, in which $Q^{1/2}Q^{1/2} = Q$, is detectable with $Q \succeq 0$, and $R \succ 0$. We did not consider these weaker assumptions since we have not discussed the notions stabilizability and detectability in detail.

Selection of the weighting matrices Q and R The matrices Q and R determine the trade-off between state regulation and control effort. The term

$$x[k]^T Q x[k]$$

penalizes deviations of the state from the origin, while

$$u[k]^T R u[k]$$

penalizes the use of control input.

Increasing the weights in Q relative to those in R places greater emphasis on reducing state deviations from the origin. This generally results in a more aggressive controller, with larger control inputs and faster regulation toward the origin. Conversely, increasing the weights in R relative to those in Q penalizes control effort more strongly, resulting in a less aggressive controller that typically regulates the state more slowly.

For a diagonal matrix

$$Q = \text{diag}(q_1, \dots, q_n),$$

a larger value of q_i places greater importance on keeping the corresponding state component $x_i[k]$ close to zero. Similarly, for

$$R = \text{diag}(r_1, \dots, r_m),$$

a larger value of r_j places a greater penalty on using the corresponding control input $u_j[k]$.

The choice of Q and R allows the designer to shape the behavior of the closed-loop system by specifying the relative importance of state regulation and control effort. It is the *relative* weighting between Q and R that determines the optimal feedback gain. In particular, multiplying both matrices by the same positive scalar, does not change the optimal feedback gain K .

A common practical choice is to take Q and R as diagonal matrices. The diagonal entries can be selected by normalizing the state and control variables according to the largest values that are considered acceptable during operation. Let $x_{i,\max}$ denote a characteristic maximum magnitude for the i th state variable and let $u_{j,\max}$ denote a characteristic maximum magnitude for the j th control input. We may then choose

$$q_i = \frac{\tilde{q}_i}{x_{i,\max}^2}, \quad i = 1, \dots, n, \quad (43)$$

and

$$r_j = \frac{\tilde{r}_j}{u_{j,\max}^2}, \quad j = 1, \dots, m, \quad (44)$$

in which $\tilde{q}_i \in (0, 1)$ and $\tilde{r}_j \in (0, 1)$ are dimensionless tuning parameters that express the relative importance assigned to each state and control input.

With this normalization, a state variable approaching its characteristic maximum value contributes approximately $\tilde{q}_i$ to the state cost, while a control input approaching its characteristic maximum contributes approximately $\tilde{r}_j$ to the control cost. This provides a convenient way of comparing variables that may have very different units and numerical scales. However, note that LQR itself does not guarantee that these acceptable values will not be exceeded along the state and control trajectories.

Computing the infinite-horizon LQR In MATLAB, the infinite-horizon LQR gain and the solution of the DARE can be computed directly using the command

```
[K,S,P] = dlqr(A,B,Q,R);
```

Here, K is the optimal feedback gain, S is the stabilizing solution of the DARE, and P contains the closed-loop eigenvalues.

In Python, the `python-control` package provides a function analogous to MATLAB's `dlqr`. The infinite-horizon LQR gain can be computed using

```
import control
```

```
K, S, E = control.dlqr(A, B, Q, R)
```

where K is the optimal feedback gain, S is the solution of the discrete algebraic Riccati equation, and E contains the closed-loop eigenvalues. The optimal control is then

$$u = -K @ x$$

and the closed-loop system matrix is

$$A_{cl} = A - B @ K$$

Alternatively, the DARE can be solved directly using SciPy:

```
from scipy.linalg import solve_discrete_are
```

```
S = solve_discrete_are(A, B, Q, R)
```

The corresponding feedback gain can then be computed as

```
import numpy as np
```

```
K = np.linalg.solve(R + B.T @ S @ B, B.T @ S @ A)
```